Materials, Methods, and Architecture

TRACE System — Technical Implementation Methodology

Contents

1 Research Methodology

This section presents the overall methodological framework underpinning the TRACE (Technical Resolution and Claims Evaluation) system. The design integrates three complementary inference paradigms—a deterministic rule engine, an XGBoost gradient-boosted ML cascade, and a large language model—into a unified six-stage processing pipeline for warranty claim analysis. The section covers the hybrid decision engine design philosophy, the pipeline architecture, the formal problem formulation, and the evaluation strategy.

1.1 Hybrid Decision Engine Design

The TRACE system employs a hybrid decision engine that integrates three complementary inference paradigms into a unified warranty claim analysis pipeline. The three paradigms are: (1) a deterministic rule engine that encodes domain expertise as first-match-wins priority rules; (2) an XGBoost gradient-boosted ML cascade that learns non-linear classification boundaries from historical claim data; and (3) a large language model (LLM) that provides semantic understanding and natural-language output formatting. The architecture follows a six-stage processing flow where each stage contributes structured signals to the final decision, with deterministic fallbacks ensuring that the system remains operational even when the LLM is unavailable.

The hybrid decision function combines outputs from all three paradigms:

$$\text{decision}(x) = \text{Combine}\big(f_{\text{LLM}}(x),\ f_{\text{rules}}(x),\ f_{\text{ML}}(x)\big) \tag{1}$$

where f_{LLM} produces a category, normalised complaint, and confidence from the LLM service; f_{rules} evaluates 9 deterministic rules and returns the first matching rule's verdict; and f_{ML} runs the XGBoost cascade to produce failure analysis and warranty decision predictions with associated probabilities. The `Combine` function is implemented as `combine_scores()` (`ml_predictor.py:664`), which performs weighted linear blending of confidence scores with conditional weights based on inter-source agreement (see Section 4.6).

The decision engine tag assignment, which identifies which paradigms contributed to the final output, is determined as follows:

$$\text{engine} = \begin{cases} \text{“LLM+Rule+ML”} & \text{if rule fires } \wedge \text{ LLM has signal} \\ \text{“Rule+ML”} & \text{if rule fires } \wedge \text{ no LLM signal} \\ \text{“LLM+ML”} & \text{if no rule fires } \wedge \text{ LLM has signal} \\ \text{“ML”} & \text{otherwise} \end{cases} \tag{2}$$

where the condition “LLM has signal” is defined as $\text{confidence}_{\text{LLM}} \geq 0.5$, using the constant `LLM_CONFIDENCE_FOR_TAG = 0.5` (`ml_predictor.py:652`). The tag assignment is implemented at lines 780–787 of `ml_predictor.py`.

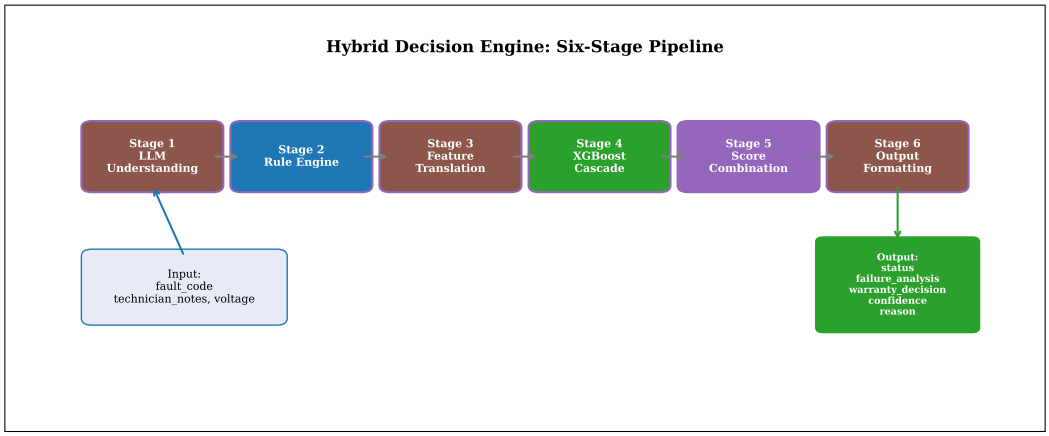

Figure 1: Hybrid decision engine overview: the six-stage processing pipeline with LLM, rule, and ML inference paths.

Design rationale: The hybrid engine is implemented in `ml_predictor.py` (941 lines) with the main entry point at `predict()` (line 836). The design philosophy prioritises deterministic rules for clear-cut cases—voltage extremes (>16 V over-voltage, <11 V under-voltage), keyword matches (moisture, physical damage, NTF), and DTC prefix patterns (U-series, P0-series, C-series, B-series)—while delegating ambiguous cases to the ML cascade. The LLM provides semantic understanding at Stage 1 and output formatting at Stage 6, with graceful degradation to rule-based and ML-only modes when API keys are unavailable. The system is served via FastAPI at `main.py:112` through the `/analyze` endpoint.

Used in: Section 1.2 (pipeline details), Section 4.6 (combination weights).

1.2 Six-Stage Pipeline Architecture

The prediction pipeline processes each warranty claim through six sequential stages, each performing a specific transformation. Stages 1, 3, and 6 are LLM-enhanced with deterministic fallbacks; Stages 2, 4, and 5 are always active regardless of LLM availability. This design ensures that the core analysis (rule evaluation, ML classification, and score combination) is never dependent on external API availability.

The pipeline composition is expressed as:

$$y = f_6\Big(f_5\big(f_4(f_3(f_2(f_1(x))))\big)\Big) \tag{3}$$

where each stage function is defined as follows:

f_1: LLM claim understanding (`llm_client.py:346`, `understand_claim()`) (4)

f_2: Rule engine evaluation (`ml_predictor.py:465`, `run_rules()`) (5)

f_3: Feature translation (`llm_client.py:437` or fallback `extract_dtc_features()`) (6)

f_4: XGBoost cascade (`ml_predictor.py:495`, `run_ml()`) (7)

f_5: Score combination (`ml_predictor.py:664`, `combine_scores()`) (8)

f_6: Output formatting (`llm_client.py:273` or fallback `assemble_output_from_fields()`) (9)

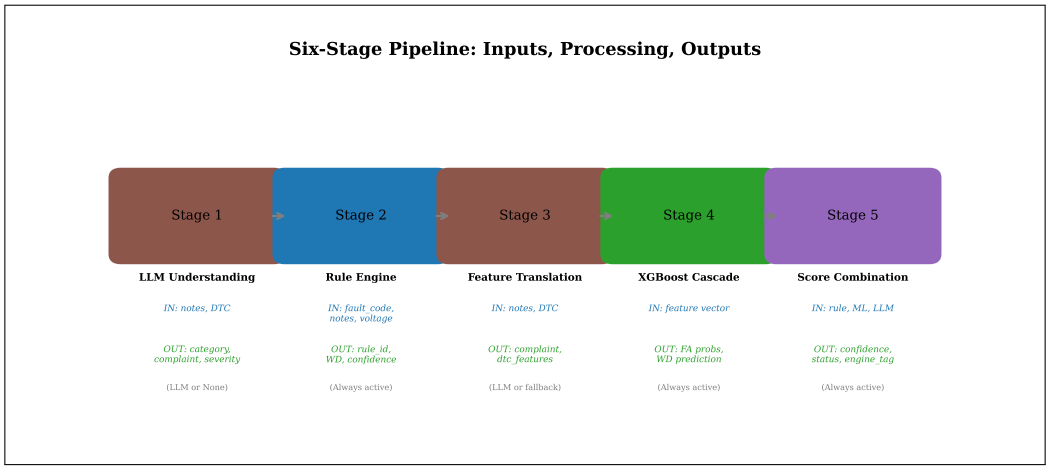

Figure 2: Six-stage pipeline architecture with input/output types and fallback paths for each stage.

The pipeline is orchestrated in the `predict()` function (`ml_predictor.py:836-925`). Each stage has explicit error handling with `try/except` blocks that trigger fallbacks. The `DecisionLogger` class (`logging_config.py:38`) logs structured output at each stage via `log_decision()` calls. LLM availability is checked at line 848:

```
api_key_available = bool(os.getenv("OPENROUTER_API_KEY"))
llm_available = api_key_available and len(notes) > 5
```

This dual condition ensures that the LLM is only invoked when both an API key is present and the technician notes contain sufficient text (more than 5 characters) for meaningful semantic analysis.

Used in: Section 5 (detailed per-stage analysis), Section 4.4 (LLM service details).

1.3 Research Problem Formulation

The TRACE system addresses the warranty claim analysis problem as a dual classification task. Given a claim described by a DTC code, free-text technician notes, and a measured voltage reading, the system must produce two outputs: (1) a Failure Analysis identifying the root cause of the component failure among 6 possible classes, and (2) a Warranty Decision determining financial responsibility among 3 possible classes. The dual-output requirement reflects the business process: the failure analysis guides technical remediation, while the warranty decision drives financial adjudication.

The Failure Analysis task is a 6-class classification problem:

$$\begin{aligned}\text{FA}\colon \mathcal{X} \to \{&\text{NTF},\ \text{Track burnt due to EOS},\ \text{ASIC CJ327 failure due to EOS},\\ &\text{Sensor short due to moisture},\ \text{Connector damage},\ \text{Controller failure}\}\end{aligned} \tag{10}$$

The Warranty Decision task is a 3-class classification problem:

$$\text{WD}\colon \mathcal{X} \to \{\text{Production Failure},\ \text{Customer Failure},\ \text{According to Specification}\} \tag{11}$$

The input space is defined as:

$$\mathcal{X} = \mathcal{D}_{\text{code}} \times \mathcal{T}_{\text{notes}} \times \mathbb{R}_{\text{voltage}} \tag{12}$$

where $\mathcal{D}_{\text{code}}$ is the set of DTC (Diagnostic Trouble Code) strings, $\mathcal{T}_{\text{notes}}$ is the space of free-text technician observations, and $\mathbb{R}_{\text{voltage}}$ is the measured voltage in volts. DTC codes follow the SAE J2012 standard format: a single-letter prefix (P for Powertrain, U for Network, C for Chassis, B for Body) followed by four hexadecimal digits.

The dual classification is handled by a cascade architecture where the FA classifier's probability vector is concatenated with the original features and fed to the WD classifier (`ml_predictor.py:427`). This allows the WD classifier to leverage the FA model's confidence distribution as an additional feature—for example, if the FA classifier assigns high probability to "ASIC CJ327 failure due to EOS," the WD classifier can use this signal alongside voltage and mileage features to determine whether the root cause warrants a Production Failure or Customer Failure classification. The 6 FA classes and 3 WD classes are defined in the synthetic dataset generator (`generate_dataset_v9(1).py`).

Used in: Section 2 (class distributions), Section 4.2 (cascade details), Section 4.1 (classifier design).

1.4 Evaluation Strategy Overview

The evaluation methodology assesses the TRACE system at three levels: (1) isolated ML classifier performance on held-out test data, (2) end-to-end pipeline accuracy including rule engine overrides and score combination, and (3) cross-validation variance estimation on unseen data. This three-tier approach ensures that both individual component quality and system-level integration correctness are measured.

Test set accuracy is computed as:

$$\text{Acc}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} \mathbb{I}(\hat{y}_i = y_i) \tag{13}$$

where $\mathbb{I}(\cdot)$ is the indicator function and $|\mathcal{D}_{\text{test}}| = 20{,}000$ (20% of the 100,000-row dataset).

Cross-validation on the held-out test set uses 3-fold partitioning:

$$\text{CV}_{\text{acc}} = \frac{1}{3} \sum_{k=1}^{3} \text{Acc}\big(\mathcal{D}_{\text{test}}^{(k)}\big) \tag{14}$$

where each fold $\mathcal{D}_{\text{test}}^{(k)}$ contains approximately 6,667 rows. Critically, this cross-validation is performed exclusively on the held-out test set—not on the training data—to provide a proper variance estimate of generalisation performance without any overlap with the training set.

End-to-end pipeline evaluation runs the complete `predict()` pipeline on sampled test rows:

$$\text{Acc}_{\text{pipeline}} = \frac{1}{N_{\text{sample}}} \sum_{i=1}^{N_{\text{sample}}} \mathbb{I}\big(\text{predict}(x_i).\text{warranty\_decision} = y_i^{\text{WD}}\big) \tag{15}$$

Evaluation is implemented in `evaluate_model.py` (528 lines). The script addresses five documented fixes from the original evaluator: (1) data leakage prevention by ensuring the train/test split occurs before any transformer fitting (FIX 1); (2) correct cross-validation on the held-out test set only (FIX 2); (3) cascade calibration checking between training and inference FA probability distributions (FIX 3); (4) end-to-end pipeline evaluation beyond isolated classifier metrics (FIX 4); and (5) preprocessing consistency between training and inference (FIX 5). The train/test split uses `test_size=0.2, random_state=42` (`ml_predictor.py:297-299`).

Used in: Section 6 (detailed evaluation metrics and results).

2 Dataset Architecture

This section describes the synthetic dataset used to train and evaluate the TRACE system, covering the dataset schema, class distributions, feature-specific distributions (voltage, mileage, DTC codes), warranty decision probability structures, and label noise injection mechanisms. Every parameter value reported in this section is drawn directly from the dataset generator source code at `generate_dataset_v9(1).py` (779 lines) and the resulting data file `synthetic_warranty_claims_v9.csv` (100,000 rows, 11 columns).

2.1 Synthetic Dataset Overview

The TRACE system is trained on a synthetically generated dataset of 100,000 warranty claims spanning model years 2019–2025, designed to mimic real-world automotive warranty data with realistic noise levels, feature correlations, and temporal drift. The dataset is not a simple random sample—it encodes domain-specific relationships between failure modes, voltage signatures, mileage patterns, DTC code distributions, and warranty adjudication decisions that reflect actual failure physics.

The dataset size and column schema are:

$$|\mathcal{D}| = 100{,}000 \text{ rows}, \quad 11 \text{ columns} \tag{16}$$

Table 1: Dataset schema for `synthetic_warranty_claims_v9.csv`. All 11 columns with data types, example values, and distribution notes.

Column	Type	Description & Distribution
Customer	categorical	7 OEMs (Ashok Leyland, M&M, Honda, Kia, Toyota, Hyundai, TATA); weighted draw
Year	integer	2019–2025; weights $[0.04, 0.07, 0.10, 0.13, 0.18, 0.23, 0.25]$
Date	date	Random date within chosen year; some classes use month weights
QC_Number	string	Sequential quality control identifier
Customer Complaint	categorical	14 complaint labels; drawn from FA-specific pools with DTC bias
DTC	string	Comma-separated DTC codes; class-specific pools with companion/cross-FA injection
Voltage	float	Truncated normal per FA class; EOS classes receive mileage-proportional nudge
Failure Analysis	categorical	6 classes: NTF, Track burnt, ASIC, Moisture, Connector, Controller
Warranty Decision	categorical	3 classes: Production Failure, Customer Failure, According to Specification
Supplier	categorical	5 suppliers (Hanon, Bosch, Valeo, Delphi, STM); class-specific weights
Mileage_km	integer	Log-normal per FA class; Connector uses bimodal mixture (W6)

The pattern correlation target for the dataset is approximately 93–96%—intentionally not 100%—to reflect real-world noise and boundary-case ambiguity. This is achieved through six label noise mechanisms (see Section 2.7) and cross-FA DTC injection (see Section 2.5).

Dataset Schema: synthetic_warranty_claims_v9.csv

100,000 rows × 11 columns

Column	Type	Example
Customer	categorical	Ashok Leyland
Year	integer	2022
Date	date	2022-06-15
QC_Number	string	QC-12345
Customer Complaint	categorical	OBD Light ON
DTC	string	P0562,P0563
Voltage	float	14.2
Failure Analysis	categorical	NTF
Warranty Decision	categorical	According to Spec
Supplier	categorical	Bosch
Mileage_km	integer	45000

Figure 3: Dataset schema overview with example rows illustrating the column structure and data types.

The dataset is generated by `generate_dataset_v9(1).py` (779 lines) and saved as `synthetic_warranty_claims_v9.csv`. Reproducibility is ensured through seeded random number generation: `rng = np.random.default_rng(42)` at line 22, and `random.seed(42)` at line 23. The generator implements 10 named improvements over previous versions (C4, C5, C6, W5, W6, W7, DTC1–DTC4), each documented in the module header (lines 1–14). The dataset is loaded at `ml_predictor.py:94`:

```
1 DATA_PATH = os.path.join(BASE_DIR, "synthetic_warranty_claims_v9.csv")
```

Used in: Section 2.2 (class proportions), Section 3.1 (loading procedure).

2.2 Failure Analysis Class Distribution

The dataset contains 6 Failure Analysis classes with year-aware proportional allocation and temporal drift modelling, where class proportions shift linearly across model years. This temporal drift models realistic industry trends: improved diagnostics reduce NTF rates over time, design improvements reduce EOS-related failures, and aging fleets increase connector damage incidence.

The base FA proportions, defined at `generate_dataset_v9(1).py:277-279`, are:

$$P(\text{NTF}) = 0.300, \quad P(\text{Track}) = 0.200, \quad P(\text{Connector}) = 0.150 \tag{17}$$

$$P(\text{ASIC}) = 0.120, \quad P(\text{Moisture}) = 0.120, \quad P(\text{Controller}) = 0.110 \tag{18}$$

The temporal drift per year from the base year 2019, defined at lines 281–283, is:

$$\Delta_{\text{NTF}} = +0.002/\text{yr}, \quad \Delta_{\text{Track}} = -0.003/\text{yr}, \quad \Delta_{\text{Connector}} = +0.004/\text{yr} \tag{19}$$

$$\Delta_{\text{ASIC}} = -0.003/\text{yr}, \quad \Delta_{\text{Moisture}} = +0.003/\text{yr}, \quad \Delta_{\text{Controller}} = -0.003/\text{yr} \tag{20}$$

The normalised proportion for year t is computed by `compute_year_counts()` (lines 286–308):

$$P_{\text{norm}}(\text{FA}_i, t) = \frac{\max(0.01,\ P_{\text{base}}(\text{FA}_i) + \Delta_i \cdot (t - 2019))}{\sum_j \max(0.01,\ P_{\text{base}}(\text{FA}_j) + \Delta_j \cdot (t - 2019))} \tag{21}$$

The floor of 0.01 prevents any class from receiving zero allocation in later years. The denominator re-normalises so that proportions sum to 1.0 for each year.

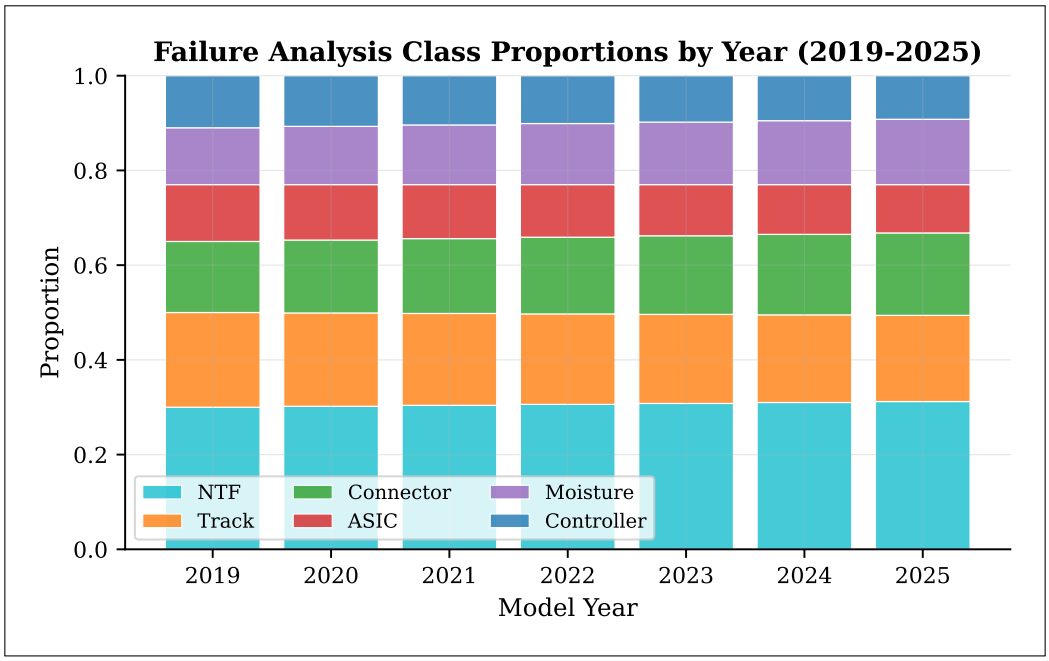

Figure 4: Failure Analysis class proportions by model year, showing temporal drift from 2019 to 2025.

The year-level row allocation is governed by year weights, defined at line 30:

$$\text{YEAR\_WEIGHTS} = [0.04, 0.07, 0.10, 0.13, 0.18, 0.23, 0.25] \tag{22}$$

corresponding to years 2019–2025. These weights reflect increasing production volumes over time, with 2025 receiving 25% of rows and 2019 receiving only 4%. The drift models the following trends: NTF claims increase over time (improved diagnostics detect more "no fault found" cases), connector damage increases (aging fleet accumulates wear), while track burnt and ASIC failures decrease (design improvements reduce electrical overstress incidence).

Table 2: Failure Analysis base proportions and annual drift rates. The "2025 projected" column shows the drift-adjusted proportion before normalisation.

FA Class	Base Proportion	Drift/yr	2025 Projected (raw)
NTF	0.300	+0.002	0.312
Track burnt due to EOS	0.200	−0.003	0.182
Connector damage	0.150	+0.004	0.174
ASIC CJ327 failure due to EOS	0.120	−0.003	0.102
Sensor short due to moisture	0.120	+0.003	0.138
Controller failure	0.110	−0.003	0.092

Used in: Section 1.3 (6-class FA task), Section 6.1 (per-class metrics).

2.3 Voltage Distribution by Failure Class

Each FA class is assigned a distinct voltage distribution using truncated normal distributions, creating separable voltage signatures that reflect real-world electrical failure modes. The truncated normal distribution ensures that generated voltages remain within physically plausible bounds while preserving the class-specific mean and variance.

The truncated normal distribution is defined as:

$$V \sim \mathcal{TN}(\mu, \sigma^2, a, b) \quad \text{with PDF:} \quad f(v) = \frac{\phi\left(\frac{v-\mu}{\sigma}\right)}{\sigma\left[\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)\right]} \tag{23}$$

where $\phi(\cdot)$ is the standard normal PDF and $\Phi(\cdot)$ is the standard normal CDF. The parameters $[a, b]$ define the truncation bounds, ensuring no values fall outside the physically valid range.

The class-specific parameters are specified directly in each generator function:

Table 3: Voltage distribution parameters by Failure Analysis class, with truncation bounds and validation assertions. All values from `generate_dataset_v9(1).py`.

Class	μ	σ	$[a, b]$	**Validation**
ASIC CJ327 failure	15.3	0.45	$[13.8, 16.5]$	mean > 15.0
Track burnt due to EOS	17.8	1.10	$[15.5, 21.0]$	mean > 17.0
Controller failure	10.4	0.60	$[8.5, 12.5]$	mean < 11.0
Sensor short (moisture)	12.7	0.55	$[10.5, 14.2]$	—
NTF	13.2	0.55	$[11.8, 14.8]$	—
Connector damage	13.3	0.65	$[11.5, 15.0]$	—

The voltage distributions are generated using `truncated_normal()` (`generate_dataset_v9(1).py:206-213`). The implementation uses the inverse CDF method: it draws a uniform random value and applies `truncnorm.ppf()` to obtain the corresponding quantile from the truncated normal distribution.

Design rationale: The EOS (Electrical OverStress) classes—ASIC CJ327 and Track burnt—receive an additional mileage-proportional voltage nudge (improvement W5, line 215):

$$V_{\text{nudged}} = V_{\text{base}} + \text{nudge\_scale} \times \frac{\text{mileage}}{200{,}000} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.03) \tag{24}$$

where `nudge_scale = 0.08` for ASIC (line 331) and `nudge_scale = 0.10` for Track burnt (line 370). This models the physical reality that higher-mileage vehicles have accumulated more electrical stress, shifting the voltage distribution upward. The nudge is applied after the base truncated normal draw and the result is clipped back to the truncation bounds. Validation assertions at lines 715–717 check: `asic_v.mean()` $>$ `15.0`, `track_v.mean()` $>$ `17.0`, `ctrl_v.mean()` $<$ `11.0`.

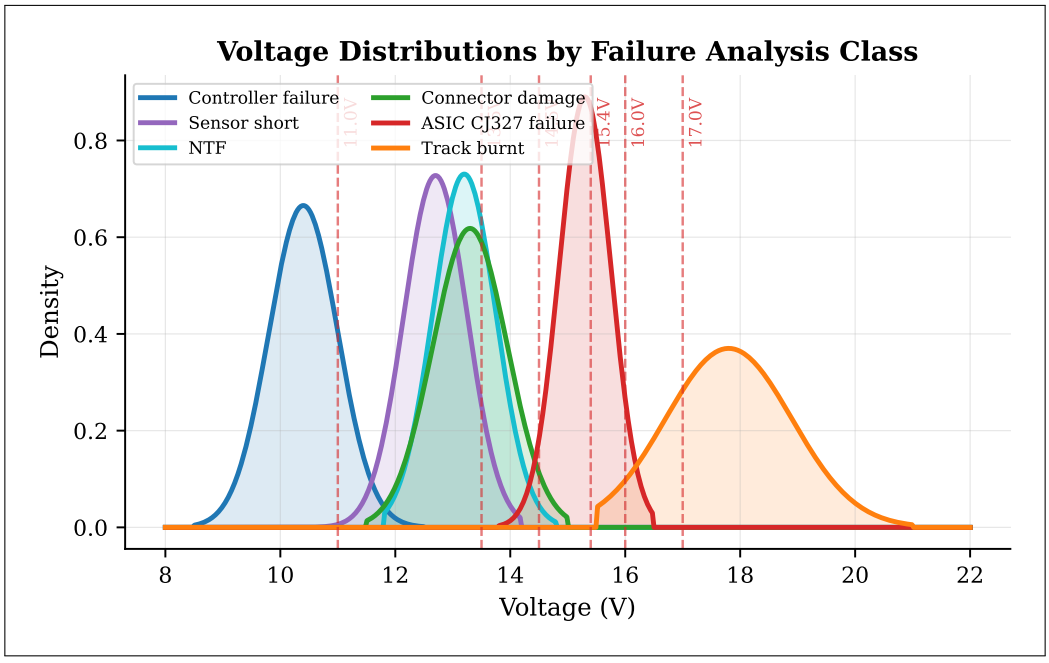

Figure 5: Voltage distributions by Failure Analysis class, showing class-specific truncated normal distributions with voltage threshold markers.

Used in: Section 4.3 (voltage-based rules), Section 3.3 (voltage bracket feature engineering).

2.4 Mileage Distribution by Failure Class

Mileage is modelled using log-normal distributions with class-specific parameters, capturing the characteristic usage patterns associated with different failure modes. The log-normal distribution naturally produces the right-skewed distributions observed in real vehicle mileage data, with a long tail of high-mileage vehicles and a concentration of claims at lower mileage values.

The log-normal distribution is defined as:

$$\text{Mileage} \sim \text{LogNormal}(\mu_{\log}, \sigma^2_{\log}), \quad \text{clipped to } [\min, \max] \tag{25}$$

where the generated value is obtained via `rng.lognormal(mu, sigma)` and then clipped using `np.clip()`.

The class-specific parameters are:

Table 4: Mileage distribution parameters by Failure Analysis class. The log-normal parameters $\mu_{\log}$ and $\sigma_{\log}$ parameterise the distribution of ln(Mileage). Clipping bounds and expected skewness are shown.

Class	$\mu_{\log}$	$\sigma_{\log}$	[min, max]	**Skewness floor**
Controller failure	ln(18,000)	0.75	[500, 90,000]	> 0.6
ASIC CJ327 failure	ln(45,000)	0.65	[3,000, 180,000]	> 0.6
NTF	ln(35,000)	0.80	[500, 220,000]	> 0.6
Sensor short (moisture)	ln(50,000)	0.70	[2,000, 200,000]	> 0.6
Track burnt due to EOS	ln(60,000)	0.65	[1,000, 220,000]	> 0.6
Connector (wear)	ln(75,000)	0.60	[15,000, 230,000]	> 0.3

The connector damage class uses a bimodal mixture distribution (improvement W6), implemented in `gen_connector_damage()` (lines 452–481):

$$\text{Mileage}_{\text{conn}} = \begin{cases} \text{LogNormal}(\ln(2{,}500),\ 0.50^2) & \text{with probability } p = 0.15 \\ \text{LogNormal}(\ln(75{,}000),\ 0.60^2) & \text{with probability } p = 0.85 \end{cases} \tag{26}$$

The early-life mode ($p = 0.15$, mean $\approx 2{,}500$ km, clipped to $[200, 8{,}000]$) represents assembly defects that manifest within the first few thousand kilometres—for example, improperly crimped connectors or misaligned pins that fail under initial thermal cycling. The wear-and-tear mode ($p = 0.85$, mean $\approx 75{,}000$ km, clipped to $[15{,}000, 230{,}000]$) represents age-related connector degradation from vibration, thermal cycling, and environmental exposure over extended service life.

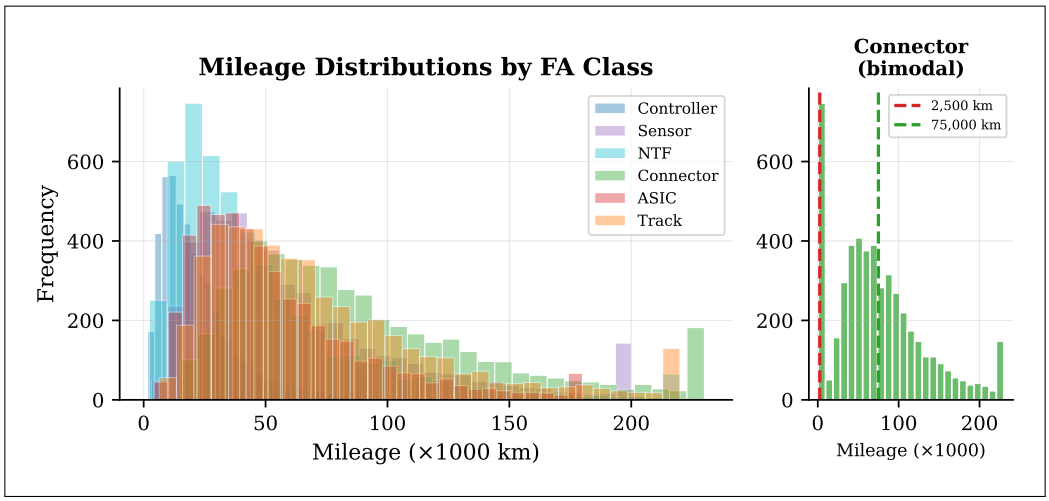

Figure 6: Mileage distributions by Failure Analysis class, with bimodal connector damage inset showing early-life assembly defects and wear-and-tear populations.

The bimodal connector distribution directly affects warranty decision probabilities: early-life assembly defects are overwhelmingly supplier responsibility ($P(\text{PF}) = 0.92$, $P(\text{CF}) = 0.08$, line 463), while wear-and-tear failures have a lower production failure rate ($P(\text{PF}) = 0.80$, $P(\text{CF}) = 0.20$, line 466). Validation

asserts mileage skewness > 0.6 for all classes except Connector (> 0.3, reflecting the bimodal mixture's reduced skewness) at lines 754–766.

Used in: Section 3.3 (mileage bracket feature), Section 2.6 (mileage-conditional WD for connector damage).

2.5 DTC Code Architecture

Diagnostic Trouble Codes (DTCs) are assigned to each FA class from native DTC pools with probabilistic companion code injection (DTC3), cross-FA code injection (DTC2), and DTC-complaint bias modelling. The DTC architecture models three real-world phenomena: (1) each failure mode produces a characteristic set of DTCs from the relevant vehicle subsystem; (2) mechanistically linked codes frequently co-occur (e.g., P0562 and P0563 are both supply voltage DTCs); and (3) some DTCs are ambiguous and appear across multiple failure modes, creating classification challenges for the ML model.

The primary DTC assignment draws uniformly from the FA-specific native pool:

$$\text{DTC}_{\text{primary}} \sim \text{Uniform}(\mathcal{P}_{\text{FA}}), \quad \mathcal{P}_{\text{FA}} \subset \text{all DTCs} \tag{27}$$

The native DTC pools per FA class are defined at `generate_dataset_v9(1).py:110-117`:

Table 5: Native DTC pools by Failure Analysis class, with code counts and representative examples. Pool sizes range from 13 (Connector) to 31 (Track).

FA Class	Pool Variable	Count	Representative Codes
ASIC CJ327 failure	`DTC_ASIC` (primary + sec.)	16	P0601, P0602, P0562, P0563, P0611, P0613, P0634
Track burnt due to EOS	`DTC_TRACK`	31	P0562, P0300–P0306, P0480–P0482, P0351–P0356
Sensor short (moisture)	`DTC_SENSOR_MOISTURE`	20	P0113, P0112, P0118, P0117, P0128, P0038
Connector damage	`DTC_CONNECTOR` + sec.	13+9	C0031, C0036, C0045, B1234, B1031, P0340, P0325
Controller failure	`DTC_CONTROLLER`	22	U0073, U0100, U0101, U0121, U0001
NTF	`DTC_NTF_MILD`	17+empty	P0455, P0456, P0171, P0420, P0430 (80% empty)

Companion DTC injection (DTC3) models mechanistically linked co-occurring codes, implemented by `maybe_append_companion_dtc()` (lines 237–247):

$$P(\text{append companion } c' \mid c) = p_{\text{companion}}(c, c') \tag{28}$$

Table 6: Companion DTC pairs with injection probabilities. Each pair represents mechanistically linked codes that frequently co-occur in real warranty data.

Primary Code	Companion Code	Injection Probability
P0562	P0563	0.55
P0691	P0692	0.60
P0616	P0617	0.50
P0351	P0352	0.45
P0601	P0604	0.50
P0602	P0606	0.45
P0605	P0607	0.40
U0100	U0101	0.60
U0073	U0001	0.55
U0121	U0140	0.45

The companion dictionary contains 13 bidirectional entries (lines 96–107). Companion injection is applied only to ASIC, Track burnt, and Controller FA classes (lines 351, 380, 503).

Cross-FA injection (DTC2) introduces ambiguous DTCs from outside the native pool, implemented by `maybe_append_cross_fa_dtc()` (lines 221–235):

$$P(\text{inject cross-FA DTC}) = 0.04 \tag{29}$$

The cross-FA injection uses `DTC_AMBIGUOUS_CROSS` (8 codes, lines 39–48: P0300, P0171, P0325, P0340, U0100, U0155, P0500, P0420) with `CROSS_FA_INJECT_RATE = 0.04` (line 49). The function pre-filters the native pool to ensure that injected codes are genuinely foreign—codes already in the FA class's native pool are excluded from injection (lines 224–225).

The ASIC primary pool was expanded from 10 to 12 codes in v9 (DTC1, lines 60–63): `DTC_ASIC_PRIMARY` now includes 12 codes (P0601, P0602, P0604, P0605, P0606, P0607, P0608, P0610, P0562, P0563, P0611, P0613), with a separate 4-code secondary pool (`DTC_ASIC_SECONDARY`: P0634, P068A, P068B, P0686). Primary codes are drawn with 88% probability of a single-code DTC; otherwise a second code is drawn from the full ASIC pool (lines 333–339).

DTC-complaint bias is modelled via `DTC_COMPLAINT_BIAS` (lines 119–181, 80+ entries), which maps DTC codes to (complaint, probability) pairs. The function `pick_complaint_with_dtc_bias()` (lines 185–192) checks whether the primary DTC has a biased complaint association; if so, it selects that complaint with the specified probability, otherwise it falls back to the FA-specific complaint pool.

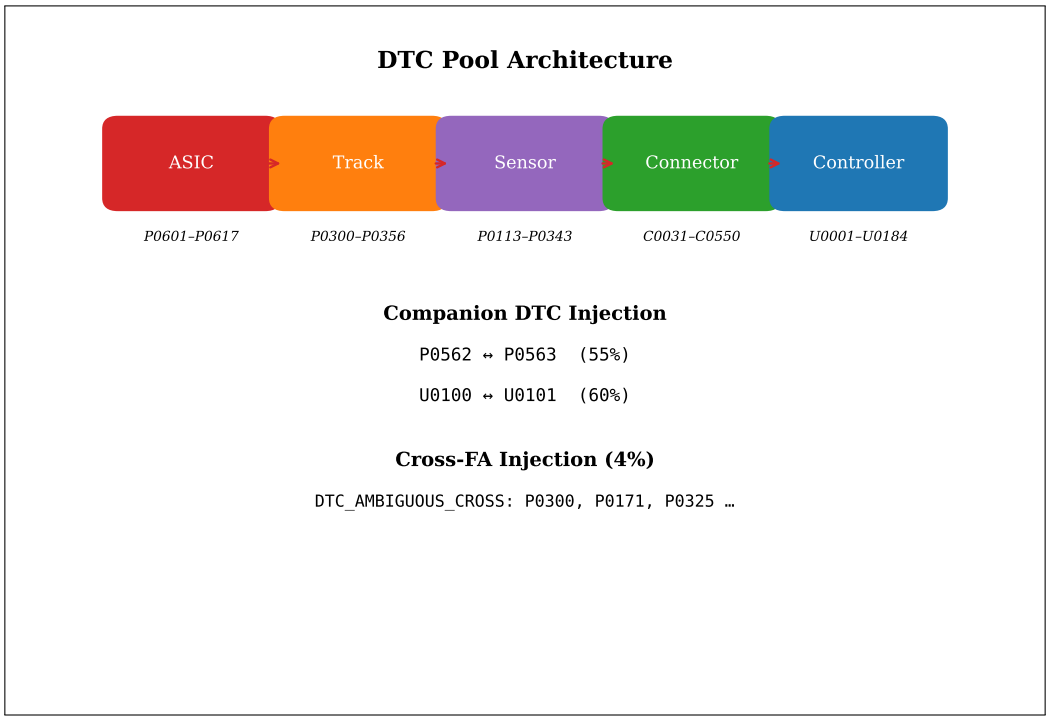

Figure 7: DTC pool architecture showing native pools per FA class, companion pair connections, and cross-FA injection pathways.

Used in: Section 3.5 (feature extraction from DTC strings), Section 4.3 (DTC-based rule matching).

2.6 Warranty Decision Probabilities

Warranty decisions are assigned probabilistically based on FA class and contextual factors (voltage thresholds, mileage), with class-specific probability distributions that reflect real-world warranty adjudication patterns. The probabilities encode the business logic that certain failure modes are predominantly supplier responsibility (Production Failure), others are customer responsibility (Customer Failure), and some represent normal operation (According to Specification).

The ASIC warranty decision is voltage-conditional, implemented at `generate_dataset_v9(1).py:344-349`:

$$P(\text{WD} \mid \text{ASIC}, V) = \begin{cases} [0.78, 0.22, 0.00] & V \leq 14.7 \\ [0.60, 0.40, 0.00] & 14.7 < V < 15.4 \\ [0.38, 0.62, 0.00] & V \geq 15.4 \end{cases} \tag{30}$$

where the probability vector is $[P(\text{PF}), P(\text{CF}), P(\text{ATS})]$. This voltage-conditional assignment reflects the physical reality that ASIC failures at lower voltages are more likely to be production defects (intrinsic component failure), while failures at higher voltages indicate electrical overstress from the vehicle's charging system (customer responsibility). The third class (ATS) has zero probability because ASIC failures are never classified as "according to specification."

The remaining FA classes use fixed or mileage-conditional probabilities:

Table 7: Warranty decision probability distributions by FA class. PF = Production Failure, CF = Customer Failure, ATS = According to Specification. The ASIC class uses voltage-conditional probabilities (Equation 30); the Connector class uses mileage-conditional probabilities (Equation 31).

FA Class	$P(\text{PF})$	$P(\text{CF})$	$P(\text{ATS})$	**Condition**
Track burnt due to EOS	0.030	0.960	0.010	Fixed (lines 375–378)
Controller failure	0.960	0.030	0.010	Fixed (lines 498–501)
NTF	0.010	0.025	0.965	Fixed (lines 438–441)
Sensor short (moisture)	0.010	0.965	0.025	Probabilistic C4 (lines 411–414)

The connector damage warranty decision is mileage-conditional, implemented at lines 461–466:

$$P(\text{WD} \mid \text{Connector}, \text{mileage}) = \begin{cases} [0.92, 0.08, 0.00] & \text{mileage} < 8{,}000 \\ [0.80, 0.20, 0.00] & \text{mileage} \geq 8{,}000 \end{cases} \tag{31}$$

The mileage threshold of 8,000 km separates the early-life assembly defect population (predominantly supplier responsibility, $P(\text{PF}) = 0.92$) from the wear-and-tear population (mixed responsibility, $P(\text{PF}) = 0.80$). This threshold directly corresponds to the bimodal mileage distribution described in Section 2.4.

Historical note: The C4 change (line 4, documented at line 410) made sensor moisture warranty decisions probabilistic rather than deterministic. In previous versions, sensor moisture was always classified as Customer Failure (100%). The v9 generator introduces a small probability of ATS ($P(\text{ATS}) = 0.025$) and PF ($P(\text{PF}) = 0.010$), better reflecting the reality that some moisture ingress occurs at the supplier level (improper sealing) and some cases resolve without confirmed failure. Validation asserts at lines 729–733: `ntf_ats_rate` $\geq$ `0.94`, `track_cf_rate` $\geq$ `0.93`, `ctrl_pf_rate` $\geq$ `0.93`.

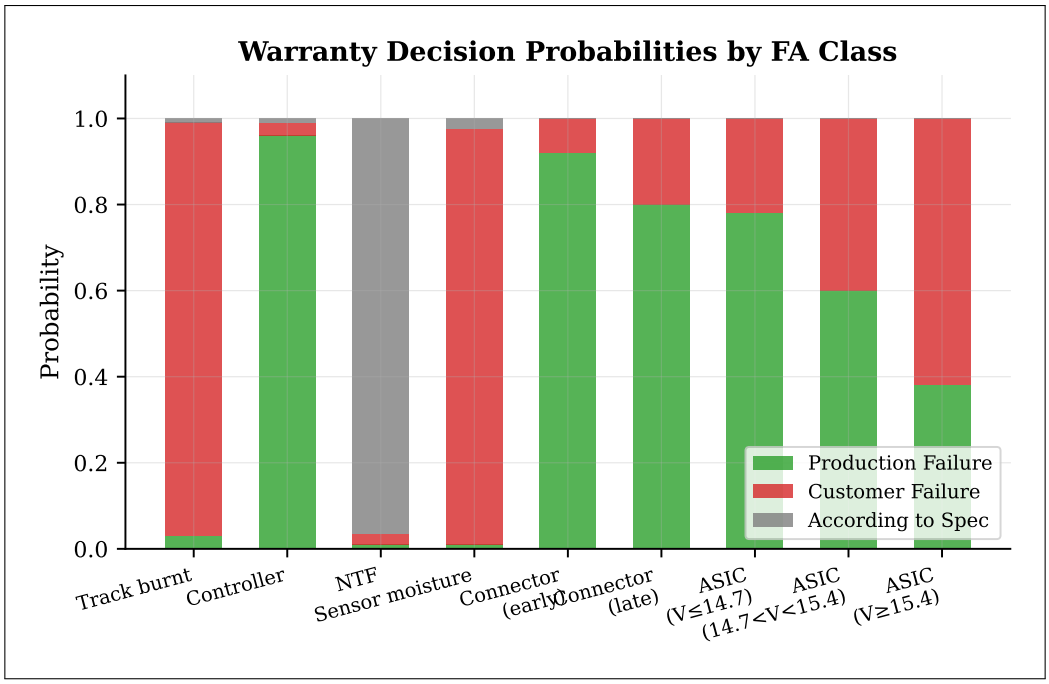

Figure 8: Warranty decision probability distributions by Failure Analysis class, showing class-specific and context-conditional probability structures.

Used in: Section 4.6 (rule confidence calibration), Section 2.7 (noise targets).

2.7 Label Noise Injection

Label noise is injected post-generation to simulate real-world annotation errors, ambiguous boundary cases, and mileage-zone uncertainty. Six distinct noise mechanisms target specific FA classes with class-appropriate flip rates. The noise is applied only to the Warranty Decision label—the Failure Analysis label is not modified—because FA represents the objective technical root cause while WD represents a potentially subjective adjudication decision.

The noise injection is implemented in `inject_warranty_label_noise()` (`generate_dataset_v9(1).py:515-595`) with a base noise rate of $\eta = 0.015$. The six mechanisms are shown in Table 8.

Table 8: Label noise injection mechanisms with target classes, flip directions, rates, and conditions. Total noise affects approximately 1.5% of rows.

Mechanism	**Target Class**	**Flip Direction**	**Rate**	**Condition**
ASIC boundary-zone noise	ASIC CJ327	PF $\leftrightarrow$ CF	$\eta \times 1.2 = 0.018$	$14.8 \leq V \leq 15.2$ (lines 519–529)
Connector random noise	Connector damage	PF $\leftrightarrow$ CF	$\eta = 0.015$	All connector rows (lines 531–540)
NTF adjudication noise	NTF	ATS $\rightarrow$ CF	0.008	All NTF rows (lines 542–549)
Track misclassification	Track burnt	CF $\rightarrow$ PF	0.007	All track rows (lines 551–558)
Controller misclassification	Controller	PF $\rightarrow$ CF	0.007	All controller rows (lines 560–567)
Sensor moisture edge	Sensor moisture	CF $\rightarrow$ PF	0.010	All moisture rows (lines 569–576)
Mileage boundary noise (W7)	All classes	PF $\leftrightarrow$ CF	0.035	88,000 $\leq$ km $\leq$ 112,000 (lines 578–591)

The ASIC boundary-zone noise is particularly significant because it targets the voltage range where the deterministic warranty decision switches from PF to CF (see Equation 30). At $V = 14.8$ to $V = 15.2$, the voltage is ambiguous—it could indicate either a production defect or customer-caused overstress—and the noise rate is elevated by a factor of 1.2 to $\eta \times 1.2 = 0.018$ (line 524).

The mileage boundary noise (W7) targets the 88,000–112,000 km zone where warranty coverage transitions from covered to expired in many real-world policies. The noise rate of 0.035 (line 580, `BOUNDARY_NOISE_RATE = 0.035`) is more than double the base rate, reflecting the heightened adjudication uncertainty in this mileage band. The boundary thresholds are defined as constants at line 579: `BOUNDARY_LO = 88_000`, `BOUNDARY_HI = 112_000`.

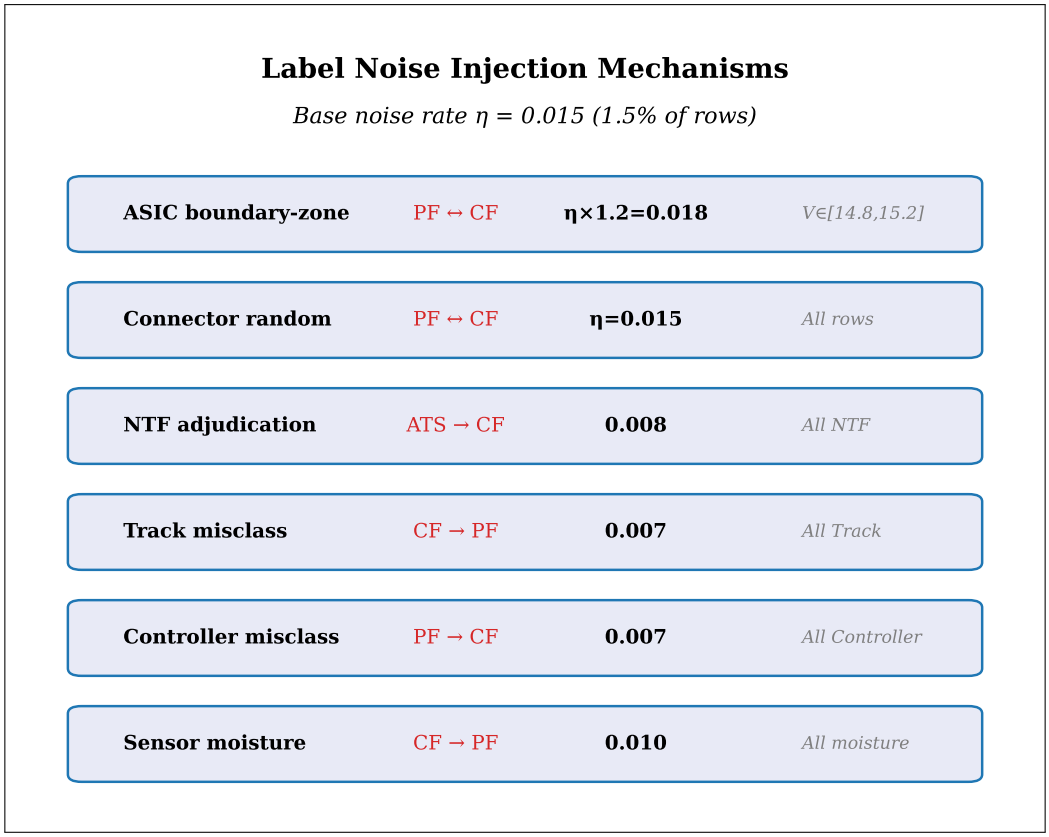

Figure 9: Label noise injection mechanisms showing target classes, flip directions, rates, and boundary conditions.

The function prints summary statistics at lines 593–594:

```
print(f"  Label noise: {total_flipped} rows flipped "
      f"({total_flipped / len(df) * 100:.2f}%)")
print(f"  Boundary zone rows: {boundary_mask.sum():,}"
      f"  |  boundary flips: {n_boundary}")
```

Total noise affects approximately 1.5% of rows across the entire dataset, providing a realistic level of label uncertainty without overwhelming the signal that the ML classifiers need to learn.

Used in: Section 6.1 (noise impact on classifier accuracy), Section 2.6 (clean WD distributions before noise).

3 Data Preparation Pipeline

This section describes the data preparation pipeline that transforms the raw CSV dataset into the feature matrices used for model training and inference. The pipeline is implemented entirely within `train_and_save()` (`ml_predictor.py:278-451`) and follows a strict ordering constraint: the train/test split occurs before any transformer fitting to prevent data leakage. The section covers data loading and cleaning, the train/test split strategy, derived feature engineering, and the transformer fitting pipeline.

3.1 Data Loading and Cleaning

The data preparation pipeline begins by loading the CSV dataset and applying consistent null-filling and normalisation to ensure all features are present and properly typed before any transformation. The null-filling strategy uses domain-appropriate defaults rather than statistical imputation, ensuring that missing values are treated as meaningful signals rather than noise.

The null handling rules, implemented at `ml_predictor.py:281-284`, are:

$$\text{DTC} \leftarrow \text{fillna("")} \text{ then replace(“none”, "")} \tag{32}$$

$$\text{Customer Complaint} \leftarrow \text{fillna(“OBD Light ON”)} \tag{33}$$

$$\text{Failure Analysis} \leftarrow \text{fillna(“NTF”)} \tag{34}$$

$$\text{Warranty Decision} \leftarrow \text{fillna(“According to Specification”)} \tag{35}$$

The DTC null handling is a two-step process: first, `fillna("")` converts actual null values to empty strings; then `replace("none", "")` normalises the string literal “none” (which appears in the dataset for NTF claims with no DTC codes) to an empty string. This ensures that both `NaN` and the string “none” are treated identically during feature extraction.

The label encoding transforms categorical target columns into integer arrays:

$$y_{\text{FA}} = \text{LabelEncoder().fit\_transform(df[“Failure Analysis”])} \tag{36}$$

$$y_{\text{WD}} = \text{LabelEncoder().fit\_transform(df[“Warranty Decision”])} \tag{37}$$

Critical: The LabelEncoders are fit on the full dataset (lines 288–289) before the train/test split. This is safe because target encoding does not expose test-set feature statistics—the encoder merely maps class labels to integers, and all 6 FA classes and 3 WD classes are guaranteed to appear in both training and test partitions given the dataset size (100,000 rows).

The null handling code is implemented at `ml_predictor.py:281-284`:

```
df["DTC"]                = df["DTC"].fillna("").replace("none", "")
df["Customer Complaint"] = df["Customer Complaint"].fillna("OBD Light ON")
df["Failure Analysis"]   = df["Failure Analysis"].fillna("NTF")
df["Warranty Decision"]  = df["Warranty Decision"].fillna("According to
    Specification")
```

The LabelEncoder fitting is done at lines 288–289:

```
le_fa = LabelEncoder(); y_fa = le_fa.fit_transform(df["Failure Analysis"])
le_wd = LabelEncoder(); y_wd = le_wd.fit_transform(df["Warranty Decision"])
```

DTC features are extracted at line 291 via `dtc_feats = pd.DataFrame(list(df["DTC"].apply(extract_dtc_features)))`. This produces a DataFrame with columns for DTC count, prefix flags (has_P, has_U, has_C, has_B), DTC text, and one-hot flags for 90+ high-value DTC codes. See Section 3.5 for details.

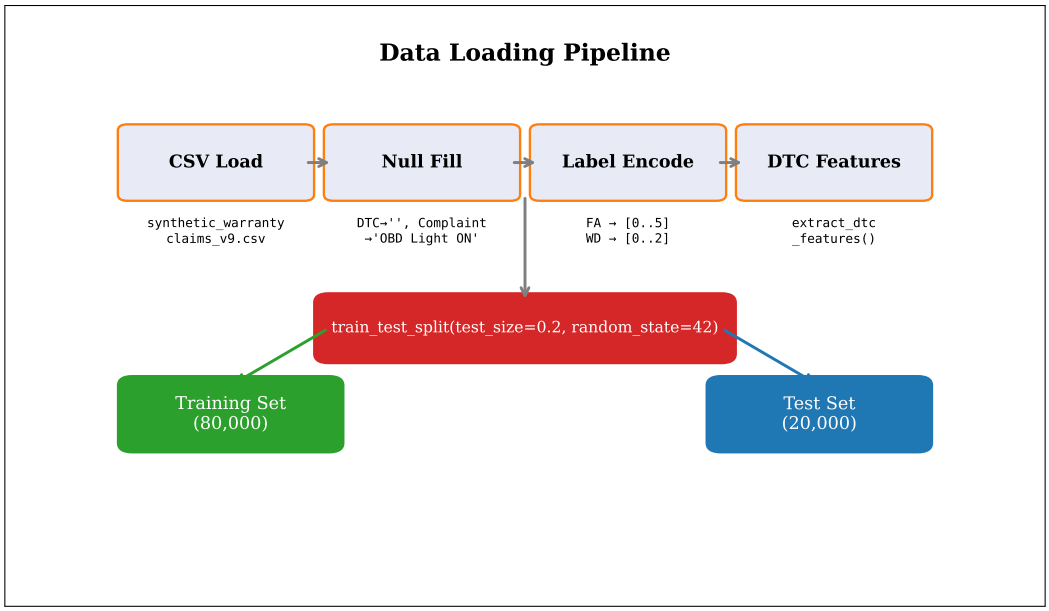

Figure 10: Data loading and cleaning pipeline showing the sequence of null handling, label encoding, DTC extraction, and train/test split.

Used in: Section 3.2 (split after cleaning), Section 3.5 (DTC extraction details).

3.2 Train-Test Split Strategy

The dataset is split into training (80%) and test (20%) sets before any transformer fitting, ensuring that no test-set statistics (vocabulary, IDF weights, mean, standard deviation) can leak into the transformers used for inference. This is the single most important data integrity constraint in the preparation pipeline.

The split configuration is:

$$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}} = \text{train\_test\_split}(\mathcal{D},\ \text{test\_size} = 0.2,\ \text{random\_state} = 42) \tag{38}$$

The split produces 8 objects simultaneously to maintain alignment across all derived arrays:

$$(\text{df}_{\text{tr}},\ \text{df}_{\text{te}},\ \text{dtc}_{\text{tr}},\ \text{dtc}_{\text{te}},\ y_{\text{fa\_tr}},\ y_{\text{fa\_te}},\ y_{\text{wd\_tr}},\ y_{\text{wd\_te}}) \tag{39}$$

where $\text{df}_{\text{tr}}, \text{df}_{\text{te}}$ are the raw DataFrames, $\text{dtc}_{\text{tr}}, \text{dtc}_{\text{te}}$ are the DTC feature DataFrames, and the remaining four arrays are the label vectors for the two classification tasks.

Critical: The split occurs at `ml_predictor.py:297-299`. The comment at lines 293–296 explicitly documents the rationale: "Splitting df before any fit_transform call ensures that no test-set statistics (vocabulary, IDF weights, mean, std) can leak into the transformers that will later be used for inference." This addresses FIX 1 from the evaluation script, which identified that the original code fitted transformers on the full dataset before splitting. The split uses `random_state=42` for reproducibility, ensuring that the same 80,000/20,000 partition is produced across all training runs.

The 8-object split is implemented at lines 297–299:

```
df_tr, df_te, dtc_tr, dtc_te, yfa_tr, yfa_te, ywd_tr, ywd_te = train_test_split
    (
    df, dtc_feats, y_fa, y_wd, test_size=0.2, random_state=42
)
```

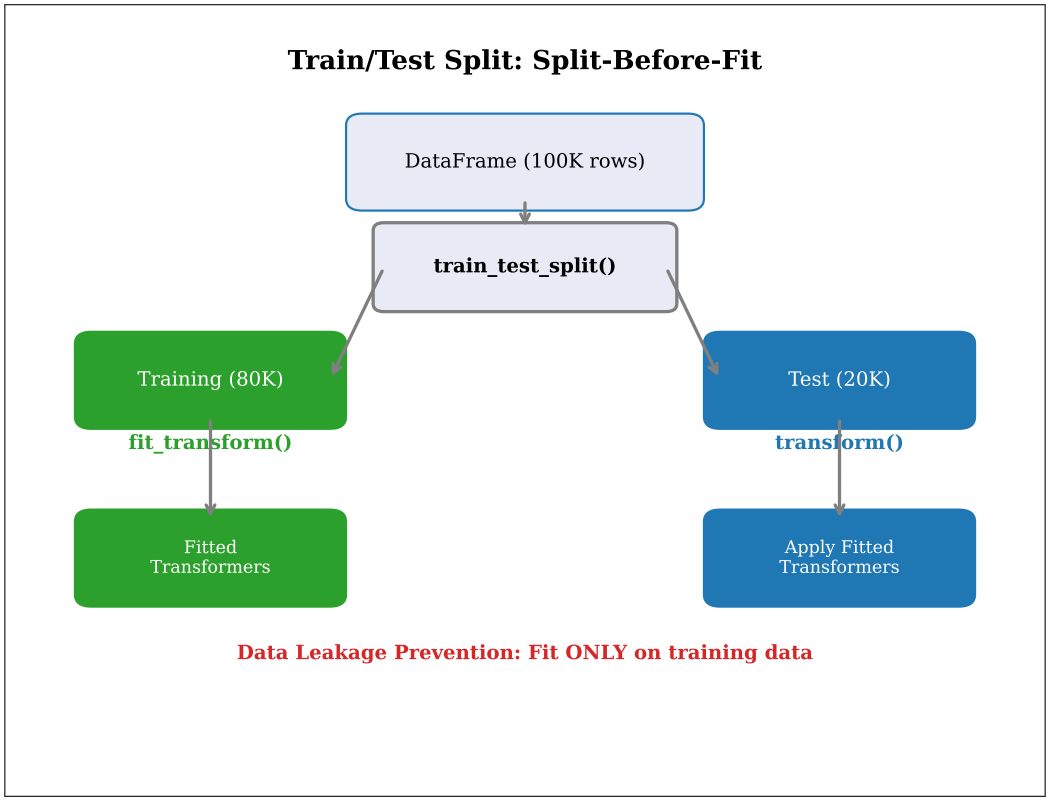

Figure 11: Train/test split strategy showing the split occurring before any transformer fitting, with separate fit/transform paths.

Used in: Section 3.4 (fitted on train only), Section 6.1 (test set evaluation).

3.3 Feature Engineering — Derived Columns

Five categories of derived features are engineered from raw columns to capture non-linear relationships, warranty eligibility signals, and cross-feature interactions that improve classifier performance. All derived features are computed after the train/test split, applied independently to both slices using the same binning logic.

1. Mileage bracket (4 bins, lines 305–312): The continuous mileage value is discretised into warranty-relevant brackets that capture the non-linear relationship between mileage and warranty eligibility better than a raw scaled value:

$$\text{mileage\_bracket} = \begin{cases} \text{“low”} & 0 \le \text{km} < 20{,}000 \\ \text{“mid”} & 20{,}000 \le \text{km} < 60{,}000 \\ \text{“high”} & 60{,}000 \le \text{km} < 100{,}000 \\ \text{“very\_high”} & \text{km} \ge 100{,}000 \end{cases} \tag{40}$$

2. Claim age (lines 318–319): The number of years between the vehicle manufacture year and the claim date provides a direct warranty-eligibility signal entirely absent from the raw “Year” column alone:

$$\text{claim\_age} = \text{year(Date)} - \text{Year} \tag{41}$$

3. Voltage bracket (7 bins, lines 323–332): The voltage value is discretised into 7 brackets aligned with the rule engine's threshold structure and the ASIC voltage-conditional warranty decision boundaries:

$$\text{voltage\_bracket} = \begin{cases} \text{“very\_low”} & V \le 11.0 \\ \text{“low”} & 11.0 < V \le 13.5 \\ \text{“normal”} & 13.5 < V \le 14.5 \\ \text{“moderate\_high”} & 14.5 < V \le 15.4 \\ \text{“high”} & 15.4 < V \le 16.0 \\ \text{“very\_high”} & 16.0 < V \le 17.0 \\ \text{“extreme”} & V > 17.0 \end{cases} \tag{42}$$

The 15.4 V threshold specifically separates the ASIC CJ327 CF vs. PF decision boundary (see Equation 30). The 16.0 V and 11.0 V boundaries align with the rule engine's over-voltage and low-voltage rules (see Section 4.3).

4. DTC count bracket (4 bins, lines 335–341): The number of DTC codes is discretised to capture the non-linear relationship between code count and failure class:

$$\text{dtc\_count\_bracket} = \begin{cases} \text{“none”} & c = 0 \\ \text{“single”} & c = 1 \\ \text{“few”} & 2 \le c \le 3 \\ \text{“many”} & c > 3 \end{cases} \tag{43}$$

5. Interaction features (4 binary, lines 344–351): These cross-feature interactions capture voltage–DTC prefix combinations that are highly predictive of specific failure modes:

$$\text{volt\_high\_and\_P} = \mathbb{I}(V > 15.4 \wedge \text{has\_P} = 1) \tag{44}$$

$$\text{volt\_low\_and\_U} = \mathbb{I}(V < 11.0 \wedge \text{has\_U} = 1) \tag{45}$$

$$\text{volt\_normal\_and\_C} = \mathbb{I}(11.0 \le V \le 14.5 \wedge \text{has\_C} = 1) \tag{46}$$

$$\text{has\_multiple\_prefixes} = \mathbb{I}(\text{has\_P} + \text{has\_U} + \text{has\_C} + \text{has\_B} > 1) \tag{47}$$

Table 9: Derived feature specifications: feature type, binning thresholds, and design rationale. Computed after the train/test split.

Feature	Type	Bins/Thresholds	Rationale
mileage_bracket	OHE (4)	20k, 60k, 100k	Non-linear warranty eligibility; OHE captures threshold sensitivity
claim_age	Scaled	Continuous	Combines vehicle year + claim date for warranty coverage signal
voltage_bracket	OHE (7)	11.0, 13.5, 14.5, 15.4, 16.0, 17.0	Aligned with rule engine thresholds and ASIC WD boundary
dtc_count_bracket	OHE (4)	0, 1, 2–3, >3	CF avg 2.3 codes, PF avg 1.5 codes; count is highly predictive
volt_high_and_P	Binary	$V > 15.4 \wedge$ has_P	ASIC/Track high-voltage EOS signal
volt_low_and_U	Binary	$V < 11.0 \wedge$ has_U	Controller low-voltage comms fault
volt_normal_and_C	Binary	$11 \leq V \leq 14.5 \wedge$ has_C	Connector in normal range
has_multiple_prefixes	Binary	>1 prefix	Multi-system failure indicator

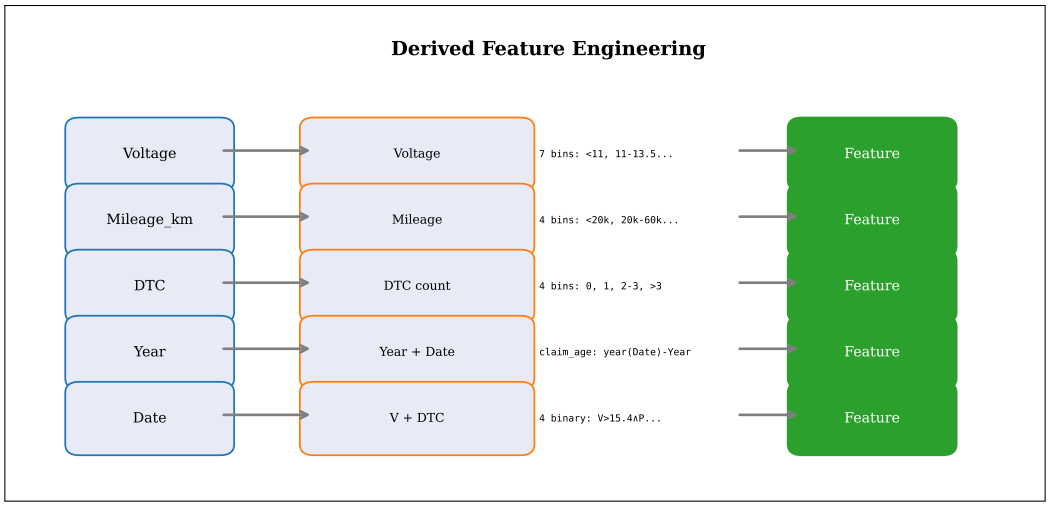

Figure 12: Derived feature engineering showing the transformation from raw columns to binned and interaction features.

The voltage bracket function is defined at lines 323–330:

```
def voltage_bracket(v):
    if v <= 11.0: return "very_low"
    elif v <= 13.5: return "low"
    elif v <= 14.5: return "normal"
    elif v <= 15.4: return "moderate_high"
    elif v <= 16.0: return "high"
    elif v <= 17.0: return "very_high"
    else: return "extreme"
```

The DTC count bracket function is defined at lines 335–341:

```
def dtc_count_bracket(c):
    if c == 0: return "none"
    elif c == 1: return "single"
    elif c <= 3: return "few"
    else: return "many"
```

The interaction features are computed at lines 344–351:

```
df_tr["volt_high_and_P"] = ((df_tr["Voltage"] > 15.4) & (dtc_tr["has_P"] == 1))
    .astype(int)
df_te["volt_high_and_P"] = ((df_te["Voltage"] > 15.4) & (dtc_te["has_P"] == 1))
    .astype(int)
df_tr["volt_low_and_U"] = ((df_tr["Voltage"] < 11.0) & (dtc_tr["has_U"] == 1)).
    astype(int)
df_te["volt_low_and_U"] = ((df_te["Voltage"] < 11.0) & (dtc_te["has_U"] == 1)).
    astype(int)
df_tr["volt_normal_and_C"] = ((df_tr["Voltage"] >= 11.0) & (df_tr["Voltage"] <=
     14.5) & (dtc_tr["has_C"] == 1)).astype(int)
df_te["volt_normal_and_C"] = ((df_te["Voltage"] >= 11.0) & (df_te["Voltage"] <=
     14.5) & (dtc_te["has_C"] == 1)).astype(int)
df_tr["has_multiple_prefixes"] = ((dtc_tr["has_P"] + dtc_tr["has_U"] + dtc_tr["
    has_C"] + dtc_tr["has_B"]) > 1).astype(int)
df_te["has_multiple_prefixes"] = ((dtc_te["has_P"] + dtc_te["has_U"] + dtc_te["
    has_C"] + dtc_te["has_B"]) > 1).astype(int)
```

Used in: Section 3.4 (transformer input), Section 5.4 (inference feature construction).

3.4 Transformer Fitting Pipeline

Twelve transformers are fitted exclusively on the training slice and then applied to both train and test slices, ensuring no data leakage. The transformers produce sparse matrices that are horizontally stacked into the final feature matrix. This "fit-on-train, transform-on-both" pattern is the standard approach for preventing information leakage from the test set into the model.

The transformer list and output specifications are:

Table 10: Transformer pipeline: 12 transformers fitted on training data only, with output types and dimensions. All OHE encoders use `handle_unknown="ignore"` for robustness to unseen categories at inference.

Transformer	Type	Output Shape	Fit On / Source
OHE (complaint)	OneHotEncoder	sparse $(n \times \lvert\mathcal{C}\rvert)$	`ohe.fit_tr(df_tr)`
TF-IDF (DTC text)	TfidfVectorizer	sparse $(n \times 40)$	`tfidf_d.fit_tr(dtc_tr)`
DTC flags	raw values	dense $(n \times (5 + \lvert\text{HV}\rvert))$	`dtc_tr[dtc_flag_cols]`
OHE (supplier)	OneHotEncoder	sparse $(n \times 5)$	`ohe_sup.fit_tr`
Scaler (mileage)	StandardScaler	dense $(n \times 1)$	`mileage_sc.fit_tr`
Scaler (year)	StandardScaler	dense $(n \times 1)$	`year_sc.fit_tr`
OHE (mileage bracket)	OneHotEncoder	sparse $(n \times 4)$	`ohe_mileage.fit_tr`
Scaler (claim age)	StandardScaler	dense $(n \times 1)$	`claim_age_sc.fit_tr`
Scaler (voltage)	StandardScaler	dense $(n \times 1)$	`voltage_sc.fit_tr`
OHE (voltage bracket)	OneHotEncoder	sparse $(n \times 7)$	`ohe_volt.fit_tr`
OHE (DTC count bracket)	OneHotEncoder	sparse $(n \times 4)$	`ohe_dtc_cnt.fit_tr`
Interactions	raw values	dense $(n \times 4)$	`df_tr[volt_hi_P, ...]`

The feature matrix assembly uses `scipy.sparse.hstack` to concatenate all 12 transformer outputs into a single sparse matrix:

$$X = \text{hstack}([X_c, X_d, X_n, X_s, X_m, X_y, X_{mb}, X_{ca}, X_v, X_{vb}, X_{dcb}, X_{int}]) \tag{48}$$

Training assembly occurs at lines 384–387 and test assembly at lines 403–406. Dense arrays (scaler outputs, DTC flags, interactions) are wrapped in `csr_matrix()` before stacking to ensure format compatibility.

Critical: The TF-IDF vectorizer uses `max_features=40` (line 360), limiting the DTC text vocabulary to the 40 most frequent terms across the training set. All OHE encoders use `handle_unknown="ignore"` (lines 359, 361–368), which silently produces all-zero vectors for unseen categories at inference time rather than raising an error. The `sparse_output=True` parameter (used by all OHE encoders) ensures memory efficiency for the one-hot encoded columns.

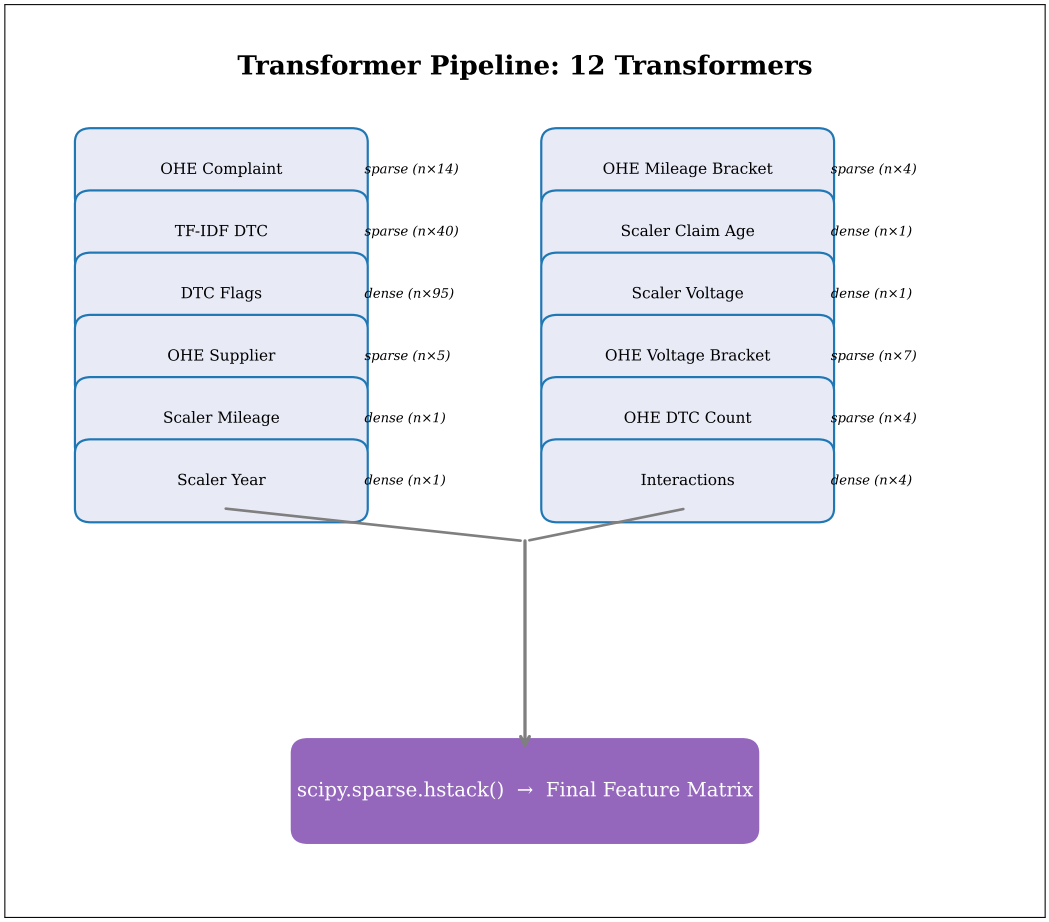

Figure 13: Transformer fitting pipeline showing the 12 transformers, their fit-on-train/transform-on-both pattern, and the horizontal stack assembly.

Used in: Section 4.1 (classifier input), Section 5.4 (inference-time feature construction).

3.5 DTC Feature Extraction

The `extract_dtc_features()` function parses comma-separated DTC code strings into structured features: prefix flags (P/U/C/B), code count, TF-IDF text, and one-hot flags for 90+ high-value DTCs. This function operates identically during both training (applied to the full DataFrame at line 291) and inference (called at line 874 for the LLM path or line 880 for the fallback path).

The DTC parsing algorithm is:

$$\text{codes} = [c.\text{strip}() \mid c \in \text{split}(\text{dtc\_str}, ","),\ c.\text{strip}() \neq ""] \tag{49}$$

The prefix flags are computed as indicator functions of the first character of each code:

$$\text{has\_P} = \mathbb{I}\big(\exists\, c \in \text{codes} :\ c.\text{startswith}(\text{“P”})\big) \tag{50}$$

with analogous definitions for has_U, has_C, and has_B (lines 237–240).

The high-value DTC one-hot encoding produces binary features for each of the 90+ codes in `HIGH_VALUE_DTCS`:

$$\text{dtc\_}\{d\} = \mathbb{I}(d \in \text{codes}), \quad \forall\, d \in \text{HIGH\_VALUE\_DTCS} \tag{51}$$

Null handling returns an all-zeros feature vector:

$$\text{if dtc\_str} \in \{"", \text{“NA”}, \text{“NAN”}, \text{“NONE”}\} \Rightarrow \text{all zeros} \tag{52}$$

The `HIGH_VALUE_DTCS` list (`ml_predictor.py:104-136`) contains 90+ codes organised by category:

Table 11: High-value DTC categories in `HIGH_VALUE_DTCS`, with code ranges and counts. Total: 90+ codes spanning all vehicle subsystems.

Category	DTC Code Ranges	Count
Misfire	P0300–P0306	7
Ignition	P0351–P0356	6
OBD processor	P0562, P0563, P0601–P0617	12
Power supply	P0620, P0691–P0694	5
Catalyst	P0420, P0430	2
EVAP	P0455–P0457	3
Vehicle speed	P0500–P0502	3
CAN bus	U0001–U0184	13
Body	B1031–B3055	5
Chassis	C0031–C0550	8
Sensor (O2/temp)	P0038–P0343	30+

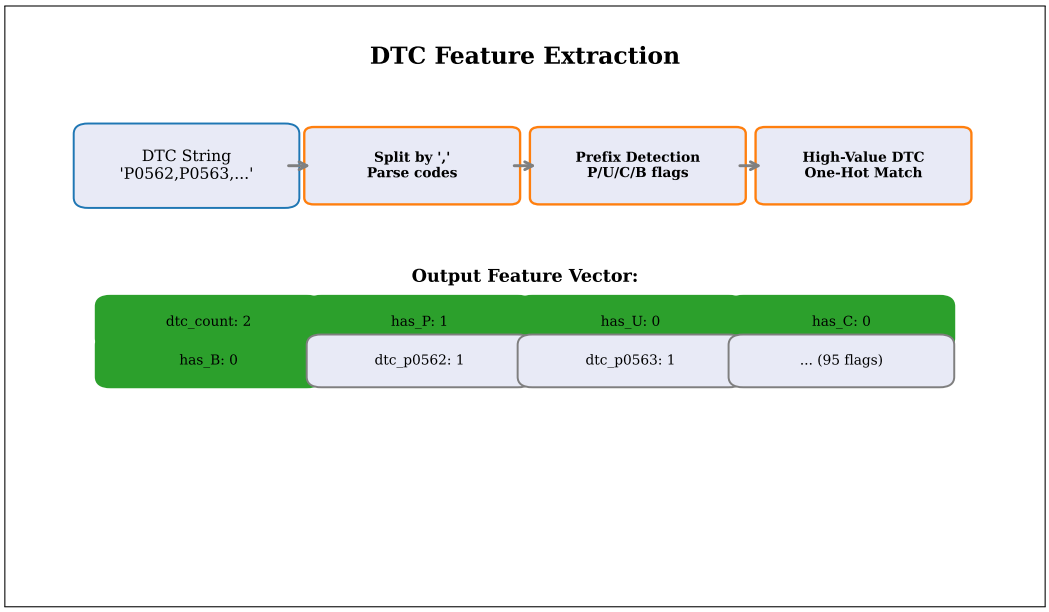

Figure 14: DTC feature extraction pipeline showing the transformation from raw DTC strings to structured feature vectors.

The complete `extract_dtc_features()` function is implemented at `ml_predictor.py:229-243`:

```python
def extract_dtc_features(dtc_str: str) -> Optional[dict]:
    s = str(dtc_str).strip().upper() if dtc_str else ""
    if s in ("", "NA", "NAN", "NONE"):
        return {"dtc_count":0,"has_P":0,"has_U":0,"has_C":0,"has_B":0,"dtc_text
            ":"none",
                **{f"dtc_{d.lower()}": 0 for d in HIGH_VALUE_DTCS}}
    codes = [c.strip() for c in s.split(",") if c.strip()]
    return {
        "dtc_count": len(codes),
        "has_P": int(any(c.startswith("P") for c in codes)),
        "has_U": int(any(c.startswith("U") for c in codes)),
        "has_C": int(any(c.startswith("C") for c in codes)),
        "has_B": int(any(c.startswith("B") for c in codes)),
        "dtc_text": " ".join(codes),
        **{f"dtc_{d.lower()}": int(d in codes) for d in HIGH_VALUE_DTCS},
    }
```

Used in: Section 5.3 (inference DTC extraction), Section 2.5 (DTC pool definitions).

3.6 Complaint Matching

The `match_complaint()` function maps free-text technician notes to one of 14 known complaint labels using a keyword-first strategy with fuzzy string matching fallback. This function is used during inference when the LLM is unavailable (fallback path at line 882) and during training to construct the complaint feature from the original dataset column.

The keyword mapping uses a first-match-wins strategy:

$$\text{complaint} = \text{value}(k) \quad \text{where } k = \text{first key in } \mathcal{K} \text{ such that } k \in \text{lowercase(notes)} \tag{53}$$

The keyword map (`kmap`, lines 250–269) contains 16 keyword-to-complaint mappings:

Table 12: Complaint keyword map used by `match_complaint()`. Keywords are checked in dictionary order (Python 3.7+ guarantees insertion order). First match wins.

Keyword	**Complaint Label**	**Keyword**	**Complaint Label**
jerk	Engine jerking accel.	abs	ABS warning light ON
accel	Engine jerking accel.	battery	Battery warning light ON
not start	Vehicle not starting	stall	Engine stalling
won't start	Vehicle not starting	multiple	Multiple warning lights
start	Starting Problem	transmission	Transmission jerking
fuel	High fuel consumption	brake	Brake warning light ON
obd	OBD Light ON	rough	Rough idling
pickup	Low pickup	idle	Rough idling
overheat	Engine overheating	warning	OBD Light ON

If no keyword matches, the fuzzy fallback uses Python's `difflib.get_close_matches`:

$$\text{complaint} = \begin{cases} \text{get\_close\_matches(notes, KNOWN,} \\ \quad n = 1,\ \text{cutoff} = 0.25)[0] & \text{if matches exist} \\ \text{“OBD Light ON”} & \text{otherwise} \end{cases} \tag{54}$$

The default complaint of "OBD Light ON" is used when the notes are empty or no keyword or fuzzy match is found. The `KNOWN_COMPLAINTS` list (`ml_predictor.py:96-102`) contains 14 complaint labels. The fuzzy matching cutoff of 0.25 is deliberately low to catch near-miss spellings and partial matches, at the cost of occasional false positives.

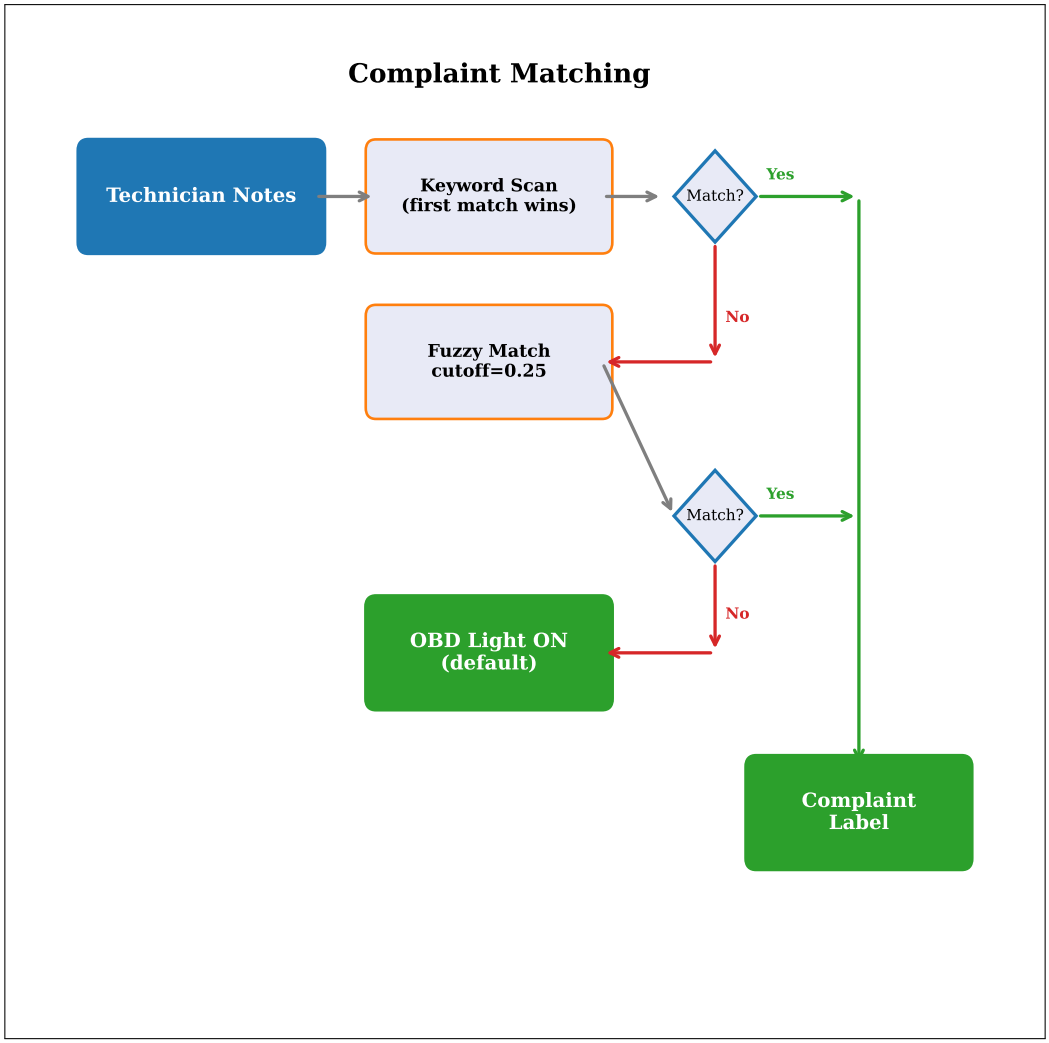

Figure 15: Complaint matching flow showing the keyword-first, fuzzy-fallback strategy with default.

The complete `match_complaint()` function is implemented at `ml_predictor.py:246-274`:

```python
def match_complaint(user_text: str) -> str:
    if not user_text:
        return "OBD Light ON"
    ul = user_text.lower()
    kmap = {
        "jerk": "Engine jerking during acceleration",
        "accel": "Engine jerking during acceleration",
        "not start": "Vehicle not starting",
        "won't start": "Vehicle not starting",
        "start": "Starting Problem",
        "fuel": "High fuel consumption",
        "obd": "OBD Light ON",
        "pickup": "Low pickup",
        "overheat": "Engine overheating",
        "idle": "Rough idling",
        "rough": "Rough idling",
        "brake": "Brake warning light ON",
        "warning": "OBD Light ON",
        "abs": "ABS warning light ON",
        "battery": "Battery warning light ON",
        "stall": "Engine stalling",
        "multiple": "Multiple warning lights ON",
        "transmission": "Transmission jerking",
    }
```

```
    for kw, complaint in kmap.items():
        if kw in ul:
            return complaint
    matches = get_close_matches(user_text, KNOWN_COMPLAINTS, n=1, cutoff=0.25)
    return matches[0] if matches else "OBD Light ON"
```

Used in: Section 5.3 (fallback feature construction).

4 Model Architecture

This section describes the three inference components of the TRACE hybrid decision engine: the XGBoost classifier configuration, the cascade architecture with out-of-fold probability generation, the deterministic rule engine, and the LLM integration architecture. Each component operates independently within the pipeline, with the score combination logic (Section 4.6) merging their outputs into a final decision.

4.1 XGBoost Classifier Configuration

The TRACE system uses XGBoost (eXtreme Gradient Boosting) classifiers as the core ML models for both the Failure Analysis and Warranty Decision tasks. Both classifiers share identical hyperparameters, configured through hyperparameter tuning and defined at `ml_predictor.py:409-413`:

$$\text{n\_estimators} = 1000 \tag{55}$$
$$\text{max\_depth} = 10 \tag{56}$$
$$\text{learning\_rate} = 0.02 \tag{57}$$
$$\text{min\_child\_weight} = 3 \tag{58}$$
$$\text{subsample} = 0.8 \tag{59}$$
$$\text{colsample\_bytree} = 0.8 \tag{60}$$
$$\text{reg\_lambda} = 0.1 \tag{61}$$
$$\text{eval\_metric} = \text{“mlogloss”} \tag{62}$$
$$\text{random\_state} = 42 \tag{63}$$

The additive model update at each boosting round is:

$$F_{t+1}(x) = F_t(x) + 0.02 \cdot h_{t+1}(x) \tag{64}$$

where $h_{t+1}(x)$ is the tree fitted to the negative gradient of the loss function at round $t+1$. The low learning rate of 0.02 combined with 1,000 estimators produces a robust ensemble that reduces overfitting through slow, incremental learning.

The regularised objective function is:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{1000} \left(0.1 \sum_{j=1}^{T_k} w_j^2 \right) \tag{65}$$

where L is the multi-class logarithmic loss, T_k is the number of leaves in tree k, and w_j is the weight of leaf j. The L2 regularisation term with $\lambda = 0.1$ penalises large leaf weights, preventing individual trees from dominating the ensemble. No L1 regularisation term ($\alpha = 0$) is used in this configuration.

Table 13: XGBoost hyperparameters with values and design rationale. Both FA and WD classifiers use identical settings.

Parameter	Value	Rationale
n_estimators	1000	Sufficient rounds for convergence with low learning rate
max_depth	10	Captures complex feature interactions without excessive overfitting
learning_rate	0.02	Slow learning for robust ensemble; prevents overfitting to noise
min_child_weight	3	Minimum samples per leaf; prevents sparse leaves
subsample	0.8	Row subsampling reduces variance per tree
colsample_bytree	0.8	Column subsampling increases tree diversity
reg_lambda	0.1	L2 regularisation on leaf weights
eval_metric	mlogloss	Multi-class log loss for early stopping
random_state	42	Reproducibility across training runs
n_jobs	−1	Parallel tree construction using all CPU cores

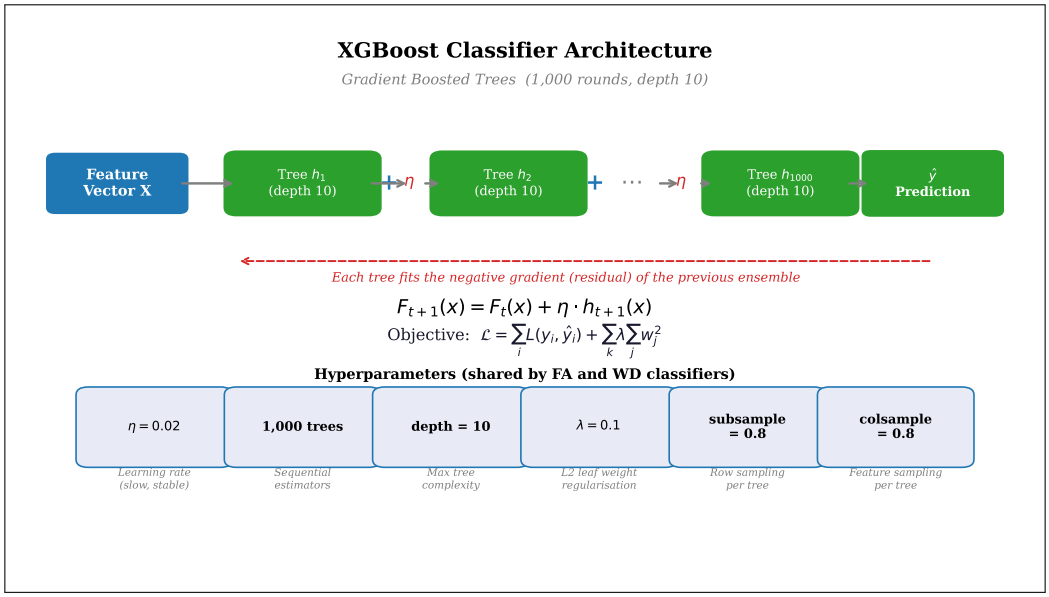

Figure 16: XGBoost architecture showing sequential tree building with learning rate scaling and subsampling.

The XGBoost hyperparameters are defined at `ml_predictor.py:409-413`:

```
_xgb_params = dict(
    n_estimators=1000, max_depth=10, learning_rate=0.02,
    min_child_weight=3, subsample=0.8, colsample_bytree=0.8,
    reg_lambda=0.1, eval_metric='mlogloss', verbosity=0, random_state=42
)
```

Used in: Section 4.2 (cascade uses two XGBoost classifiers), Section 5.4 (inference execution).

4.2 Cascade Architecture — Out-of-Fold Probabilities

The cascade architecture uses out-of-fold (OOF) probabilities from the FA classifier as additional input features for the WD classifier, preventing data leakage that would occur if the FA classifier scored its

own training data. This is a standard stacking technique adapted for the dual-classification nature of the TRACE task.

The OOF probability generation uses 5-fold cross-validation on the training set:

$$\mathbf{p}_i^{\text{OOF}} = f_{\theta_{-k}}^{\text{FA}}(x_i), \quad \text{where } i \in \mathcal{D}_k,\ k = 1, \ldots, 5 \tag{66}$$

where θ_{-k} denotes the model parameters trained on all folds except fold k, ensuring that the probability for sample i was generated by a model that never "saw" that sample during training. This is implemented at `ml_predictor.py:417-422`:

```
fa_probs_tr = cross_val_predict(
    XGBClassifier(n_jobs=-1, **_xgb_params),
    X_tr, yfa_tr,
    cv=5,
    method="predict_proba",
)
```

After generating OOF probabilities, the FA classifier is retrained on the full training data at lines 423–424:

$$\text{clf\_fa} = \text{XGBClassifier}(\ldots).\text{fit}(X_{\text{train}}, y_{\text{FA\_train}}) \tag{67}$$

The WD training feature matrix is augmented with the OOF FA probability vector:

$$X_{\text{WD}}^{\text{train}} = \text{hstack}([X_{\text{train}},\ \mathbf{p}^{\text{OOF}}]) \tag{68}$$

At inference time, test FA probabilities are obtained from the fully retrained FA classifier (line 426):

$$X_{\text{WD}}^{\text{test}} = \text{hstack}([X_{\text{test}},\ f_\theta^{\text{FA}}(X_{\text{test}})]) \tag{69}$$

The WD prediction selects the class with the highest probability:

$$\hat{y}_{\text{WD}} = \arg\max_k f_{\theta_{\text{WD}}}(X_{\text{WD}}^{\text{test}})_k \tag{70}$$

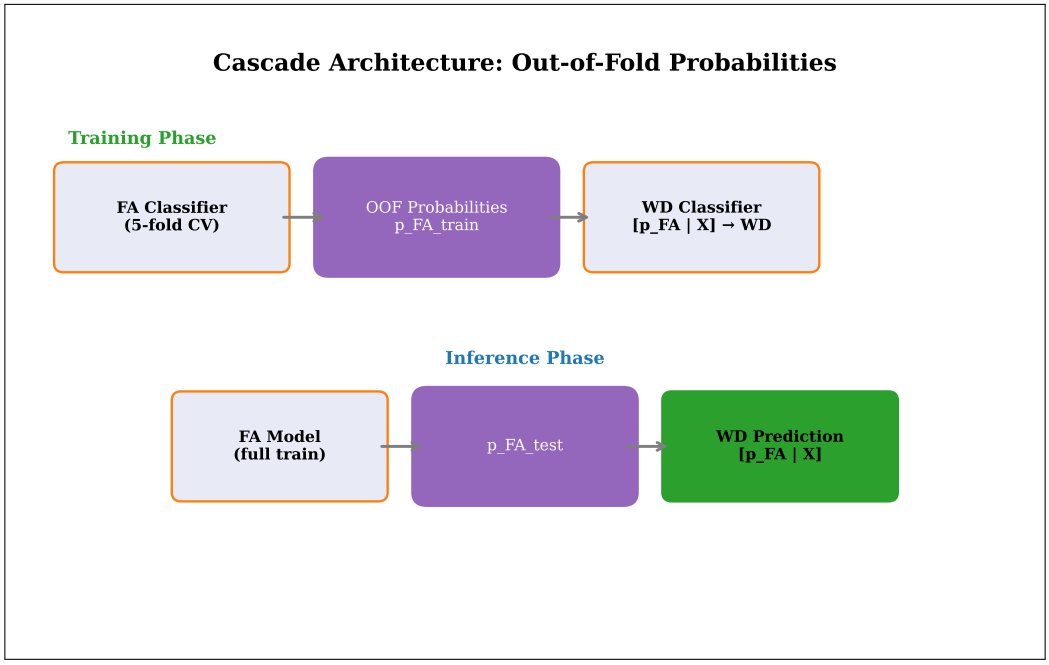

Figure 17: Cascade architecture showing OOF probability generation for training and direct prediction for inference, both feeding into the WD classifier.

Historical note: This addresses FIX 3 from the evaluation script. The original code used `clf_fa.predict_proba(X_tr)` which produced in-sample (overconfident) probabilities because the FA

classifier was scoring the same data it was trained on. The OOF approach produces calibrated probabilities that match the distribution the WD classifier will see at inference time, as verified by the cascade calibration check (Section 6.3).

Used in: Section 5.4 (inference execution), Section 6.3 (calibration verification).

4.3 Rule Engine — Complete Rule Specification

The rule engine consists of 9 deterministic rules evaluated in priority order using a first-match-wins strategy. Each rule specifies a matching condition (a predicate over fault code, technician notes, and voltage), a failure analysis, a warranty decision, a status, a confidence, and a human-readable reason. The rule engine provides high-confidence decisions for cases where domain expertise yields an unambiguous verdict, freeing the ML classifier to focus on subtler, boundary-case claims.

The rule evaluation logic, implemented in `run_rules()` (`ml_predictor.py:465-492`), is:

$$\text{result}(x) = \begin{cases} R_1 & \text{if } \phi_1(x) \\ R_2 & \text{if } \neg\phi_1(x) \wedge \phi_2(x) \\ \vdots & \\ R_9 & \text{if } \bigwedge_{i=1}^{8} \neg\phi_i(x) \wedge \phi_9(x) \\ \text{None} & \text{if } \bigwedge_{i=1}^{9} \neg\phi_i(x) \end{cases} \tag{71}$$

where $\phi_i(x)$ is the matching predicate for rule i and R_i is the corresponding result. The function iterates through the `RULES` list (lines 138–226) and returns on the first match. Each rule evaluation is wrapped in a `try/except` block (lines 480–490) to handle potential errors in individual rule predicates without aborting the entire rule evaluation.

The complete rule specification is:

Table 14: Complete rule specification for the 9 deterministic rules, evaluated in priority order with first-match-wins logic. Source: `ml_predictor.py:138-226`.

#	Rule ID	Match Condition	WD	Status	Conf. (%)
1	`over_voltage`	$V > 16.0$	CF	Rejected	93.0
2	`low_voltage`	$V < 11.0$	CF	Rejected	95.0
3	`moisture`	keyword ∈ {water, moisture, wet, flood, rain, humid, corrosion, corroded}	CF	Rejected	91.0
4	`physical_damage`	keyword ∈ {crack, broken, impact, collision, bent, misuse, dropped, phys. damage}	CF	Rejected	88.5
5	`ntf`	keyword ∈ {no fault, ntf, no trouble, no issue, no defect, intermittent, cannot reproduce}	ATS	Approved	95.0
6	`u_code`	DTC matches `\bU[0-9A-Fa-f]{4}\b`	PF	Approved	57.0
7	`p_code_engine`	DTC matches `\bP0[0-9]{3}\b` **and** keyword ∈ {jerk, pickup, accel., overheat, fuel, idle, rough}	PF	Approved	80.5
8	`c_code`	DTC matches `\bC[0-9A-Fa-f]{4}\b`	PF	Approved	80.0
9	`b_code`	DTC matches `\bB[0-9A-Fa-f]{4}\b`	PF	Approved	80.0

Design rationale: The rule ordering reflects a deliberate priority structure. Voltage rules (1, 2) come first because voltage extremes provide unambiguous evidence of charging system failure. Keyword-based rules (3, 4, 5) follow because explicit mentions of moisture, physical damage, or no-trouble-found in technician notes are highly reliable signals. DTC prefix rules (6–9) are evaluated last because DTC codes alone provide less specific evidence—for example, a U-code could indicate either a genuine controller failure or a transient communication glitch. The `u_code` rule has the lowest confidence (57.0%) reflecting this ambiguity, while the `p_code_engine` rule (80.5%) requires both a P0-series DTC *and* matching symptom keywords, increasing its reliability.

Rule confidences were recalibrated for v9's noisy patterns. The voltage and keyword rules maintain high confidences (>88%) because their triggering conditions are inherently unambiguous. The DTC prefix rules carry lower confidences (57–80.5%) because DTC codes alone do not uniquely determine the failure mode.

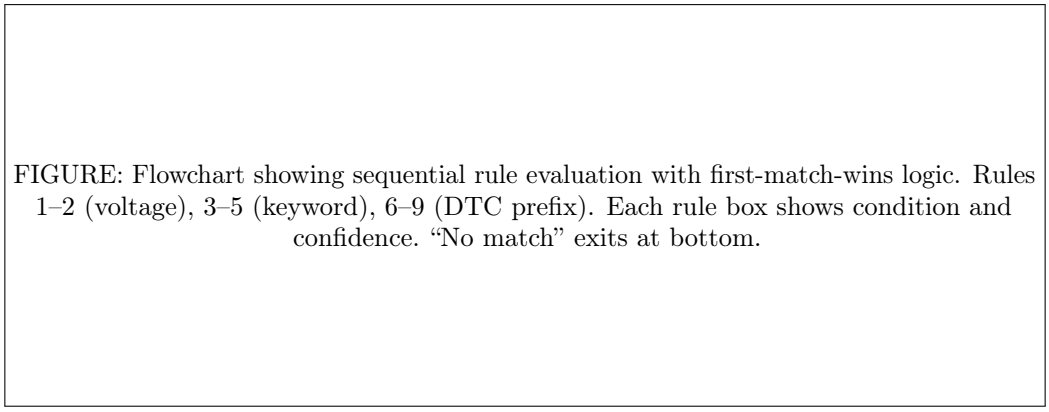

Figure 18: Rule engine flow showing sequential evaluation of 9 rules with first-match-wins priority logic.

The complete `run_rules()` function is implemented at `ml_predictor.py:465-492`:

```python
def run_rules(fault_code: str, notes: str, voltage: float = None) -> Optional[
    dict]:
    """
    Run the rule engine against the claim inputs.

    Args:
        fault_code: DTC code(s)
        notes: Technician's free-text notes
        voltage: Measured voltage (optional)

    Returns:
        dict with keys: rule_id, status, warranty_decision, rule_confidence,
            failure_analysis, reason, rule_fired
        None if no rule matches
    """
    for rule in RULES:
        try:
            if rule["match"](fault_code, notes, voltage):
                return {
                    "rule_id": rule["id"],
                    "status": rule["status"],
                    "warranty_decision": rule["warranty_decision"],
                    "rule_confidence": rule["confidence"],
                    "failure_analysis": rule["failure_analysis"],
                    "reason": rule["reason"],
                    "rule_fired": True,
                }
        except Exception:
            continue
    return {"rule_fired": False}
```

Used in: Section 5.2 (runtime execution), Section 4.6 (rule confidence in combination).

4.4 LLM Integration Architecture

The LLM integration provides three services within the six-stage pipeline: (1) claim understanding and categorisation at Stage 1, (2) feature translation at Stage 3, and (3) output formatting at Stage 6. Each service supports both OpenAI (`gpt-4o-mini`) and OpenRouter (`arcee-ai/trinity-large-preview:free`) providers with automatic fallback between them. The LLM is never mandatory—the system operates correctly in ML+Rule mode when no API key is configured.

The provider selection logic, implemented at `llm_client.py:28-33`, is:

$$\text{provider} = \begin{cases} \text{“openai”} & \text{if OPENAI\_API\_KEY is set} \\ \text{“openrouter”} & \text{if OPENROUTER\_API\_KEY is set} \\ \text{None} & \text{otherwise} \end{cases} \tag{72}$$

OpenAI is preferred when both keys are available because `gpt-4o-mini` provides more consistent JSON output formatting. The OpenRouter provider serves as a free-tier fallback.

Retry logic with exponential backoff handles transient API failures, implemented in `understand_claim_with_retry()` (`llm_client.py:380-405`):

$$\text{delay}_k = 2^k \text{ seconds}, \quad k \in \{0, 1\}, \quad \text{max\_retries} = 2 \tag{73}$$

The function attempts up to 2 calls with exponential backoff: on first failure, it waits $2^0 = 1$ second before retrying; on second failure, it waits $2^1 = 2$ seconds. If both attempts fail, it returns `None`, which triggers the fallback path in the prediction pipeline.

The LLM availability check at inference time is:

$$\text{llm\_available} = \text{api\_key\_available} \wedge \text{len(notes)} > 5 \tag{74}$$

This dual condition ensures the LLM is only invoked when both an API key is present and the technician notes contain sufficient text (more than 5 characters) for meaningful semantic analysis. Very short notes (e.g., “ok” or “fine”) provide insufficient context for LLM categorisation.

Table 15: LLM prompt services: three distinct prompts for the three LLM-enhanced pipeline stages. All prompts request JSON output with `temperature=0, seed=42` for determinism.

Service	**Function**	**Prompt Var.**	**Output Keys**
Claim understanding	`understand_claim()` (l. 346)	`UNDERSTAND_CLAIM_PROMPT` (l. 306–343)	category, normalised_complaint, severity, failure_analysis, reasoning, confidence
Feature translation	`translate_to_ml_features()` (l. 437)	`TRANSLATE_ML_FEATURES_PROMPT` (l. 408–434)	customer_complaint, dtc_codes, dtc_text, dtc_count, has_P/U/C/B
Output formatting	`format_output()` (l. 273)	`FORMAT_OUTPUT_PROMPT` (l. 243–270)	status, failure_analysis, warranty_decision, confidence, reason, matched_complaint, decision_engine

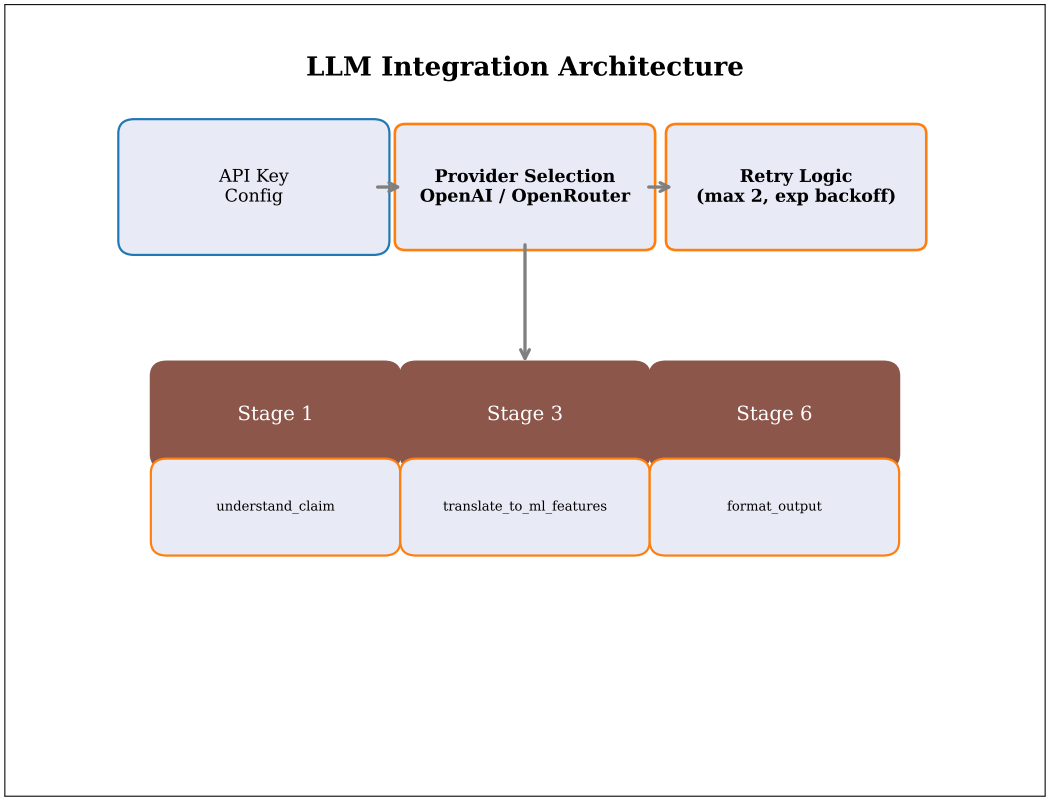

Figure 19: LLM integration architecture showing dual provider support, retry logic, and three service endpoints.

Used in: Section 4.5 (Stage 1 details), Section 5.3 (Stage 3 details), Section 5.6 (Stage 6 details).

4.5 LLM Claim Categorisation

The Stage 1 LLM service categorises warranty claims into one of 7 semantic categories using a structured prompt with disambiguation rules, producing a category, normalised complaint, severity assessment, preliminary failure analysis, reasoning, and confidence score. This categorisation occurs before any rule or ML logic runs, providing a semantic "first impression" that informs downstream score combination.

The 7 categories and their mapping to warranty decisions, defined at `ml_predictor.py:634-642`, are:

$$\text{WD}_{\text{LLM}} = \begin{cases} \text{“Customer Failure”} & \text{if cat.} \in \{\text{moisture\_damage, physical\_damage}\} \\ \text{“According to Specification”} & \text{if category} = \text{ntf} \\ \text{“Production Failure”} & \text{if cat.} \in \{\text{electrical\_issue,} \\ & \quad \text{engine\_symptom, communication\_fault}\} \\ \text{None} & \text{if category} = \text{other} \end{cases} \tag{75}$$

The "other" category maps to `None`, meaning the LLM provides no warranty decision signal for claims it cannot confidently categorise. This prevents the LLM from injecting noise into the score combination when the claim is genuinely ambiguous.

The disambiguation rules, embedded in the `UNDERSTAND_CLAIM_PROMPT` (`llm_client.py:306-343`), are applied in first-match-wins order:

1. **Engine symptom:** overheating, jerking, pickup, acceleration, fuel, idle, rough → `engine_symptom`
2. **Communication fault:** CAN bus, LIN bus, communication, network, U-code → `communication_fault`
3. **Moisture damage:** moisture, water, wet, flood, rain, humidity, corrosion → `moisture_damage`
4. **Physical damage:** crack, broken, impact, collision, bent, misuse, dropped → `physical_damage`

5. **NTF:** no fault, ntf, no trouble, no issue, no defect, intermittent → `ntf`
6. **Electrical issue:** electrical short, wiring (without engine symptoms) → `electrical_issue`
7. **Otherwise:** → `other`

Table 16: LLM category to warranty decision mapping. The "other" category provides no signal, preventing noise injection into score combination.

Category	Warranty Decision	Semantic Basis
moisture_damage	Customer Failure	Environmental contamination is customer responsibility
physical_damage	Customer Failure	Mechanical damage is customer responsibility
ntf	According to Specification	No fault found means vehicle operates correctly
electrical_issue	Production Failure	Electrical defects indicate supplier manufacturing fault
engine_symptom	Production Failure	Engine symptoms indicate component-level production fault
communication_fault	Production Failure	Communication faults indicate ECU/controller production fault
other	None	Insufficient signal; claim is genuinely ambiguous

The `understand_claim()` function (`llm_client.py:346-377`) sends the prompt with the claim's technician notes and DTC code, then parses the JSON response into a structured dictionary. The function uses default values for any missing keys: `category="other"`, `normalized_complaint="OBD Light ON"`, `severity="moderate"`, `failure_analysis="Unknown"`, `reasoning=""`, `confidence=0.0` (lines 368–374). The retry wrapper `understand_claim_with_retry()` (lines 380–405) adds the exponential backoff described in Section 4.4.

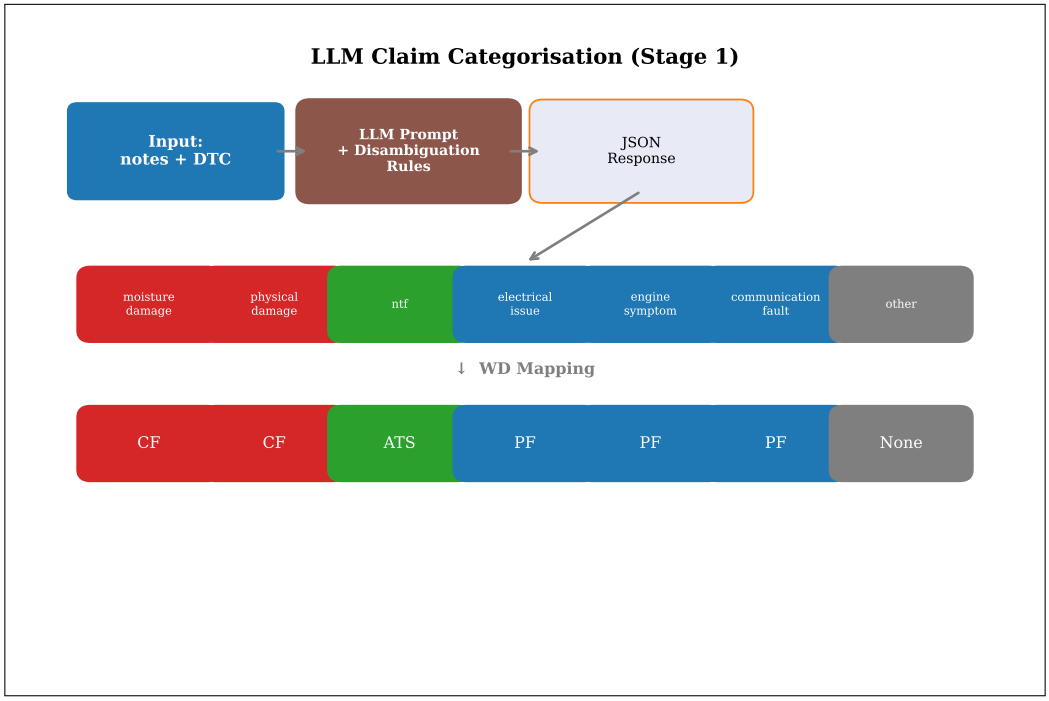

Figure 20: LLM claim categorisation flow showing the 7-category output with disambiguation rules and warranty decision mapping.

The provider selection logic is implemented at `llm_client.py:28-33`:

```python
def _get_provider() -> Optional[str]:
    if os.getenv("OPENAI_API_KEY"):
        return "openai"
    if os.getenv("OPENROUTER_API_KEY"):
        return "openrouter"
    return None
```

The LLM dispatch function is implemented at `llm_client.py:63-77`:

```python
def _call_llm(prompt: str, timeout: int = 30) -> Optional[str]:
    provider = _get_provider()
    if not provider:
        logger.error("No LLM provider configured")
        raise RuntimeError(
            "No LLM provider configured. Set OPENAI_API_KEY or
                OPENROUTER_API_KEY"
        )

    t0 = time.monotonic()

    if provider == "openai":
        return _call_openai(prompt, timeout, t0)
    else:
        return _call_openrouter(prompt, timeout, t0)
```

Used in: Section 5.1 (runtime execution), Section 4.6 (LLM agreement in combination).

4.6 Score Combination Logic

The score combiner merges rule engine confidence, ML confidence, and LLM confidence using weighted linear blending with conditional weights based on inter-source agreement. The combination logic is implemented in `combine_scores()` (`ml_predictor.py:664-808`) and represents the final decision-making step of the hybrid engine.

The constants governing score combination are defined at `ml_predictor.py:623-652`:

Table 17: Score combination constants with values and descriptions. Source: `ml_predictor.py:623-652`.

Constant	Value	Description
`CONFIDENCE_THRESHOLD_FIRM`	85.0	Confidence $\geq 85\%$ yields "Firm" decision (Approved/Rejected)
`CONFIDENCE_THRESHOLD_MANUAL`	65.0	Confidence $< 65\%$ yields "Needs Manual Review"
`AGREEMENT_BONUS`	5.0	Bonus added when rule and ML agree
`DISAGREEMENT_GAP_THRESHOLD`	20.0	Gap between rule and ML confidence for disagreement classification
`WEAK_INPUT_PENALTY`	0.85	Factor applied when no rule fires (legacy; not currently used)
`RULE_WEIGHT_AGREE`	0.70	Rule weight when rule + ML agree, no LLM signal
`ML_WEIGHT_AGREE`	0.30	ML weight when rule + ML agree, no LLM signal
`RULE_WEIGHT_DISAGREE`	0.55	Rule weight when rule and ML disagree, no LLM signal
`ML_WEIGHT_DISAGREE`	0.35	ML weight when rule and ML disagree, no LLM signal
`LLM_WEIGHT`	0.15	LLM weight in all combination scenarios
`RULE_WEIGHT_AGREE_LLM`	0.595	Rule weight when rule + ML agree, with LLM ($= 0.70 \times 0.85$)
`ML_WEIGHT_AGREE_LLM`	0.255	ML weight when rule + ML agree, with LLM ($= 0.30 \times 0.85$)
`RULE_WEIGHT_DISAGREE_LLM`	0.4675	Rule weight when rule + ML disagree, LLM agrees rule ($= 0.55 \times 0.85$)
`ML_WEIGHT_DISAGREE_LLM`	0.2975	ML weight when rule + ML disagree, LLM agrees ML ($= 0.35 \times 0.85$)
`LLM_LOW_CONFIDENCE_THRESHOLD`	0.3	Below this, LLM signal is considered weak
`LLM_LOW_CONFIDENCE_PENALTY`	0.7	Multiplier applied when LLM confidence < 0.3 and no rule fires
`LLM_CONFIDENCE_FOR_TAG`	0.5	Minimum LLM confidence for "LLM" tag in decision engine label

The combination formulas for each scenario are:
Scenario A: Rule fires AND agrees with ML (with LLM agreement):

$$c_{\text{combined}} = 0.595 \cdot c_{\text{rule}} + 0.255 \cdot c_{\text{ml}} + 0.15 \cdot (c_{\text{llm}} \times 100) + 5.0 \tag{76}$$

Scenario A: Rule fires AND agrees with ML (without LLM agreement):

$$c_{\text{combined}} = 0.70 \cdot c_{\text{rule}} + 0.30 \cdot c_{\text{ml}} + 2.0 \tag{77}$$

Scenario B: Rule fires BUT disagrees with ML (with LLM agreement):

$$c_{\text{combined}} = 0.4675 \cdot c_{\text{rule}} + 0.2975 \cdot c_{\text{ml}} + 0.15 \cdot (c_{\text{llm}} \times 100) \tag{78}$$

Scenario B: Rule fires BUT disagrees with ML (no LLM agreement):

$$c_{\text{combined}} = 0.55 \cdot c_{\text{rule}} + 0.35 \cdot c_{\text{ml}} \tag{79}$$

Scenario C: No rule fires:

$$c_{\text{combined}} = 0.85 \cdot c_{\text{ml}} + 0.15 \cdot (c_{\text{llm}} \times 100) \tag{80}$$

With weak input penalty: if $c_{\text{llm}} < 0.3$ in Scenario C, then $c_{\text{combined}} \leftarrow c_{\text{combined}} \times 0.7$ (implemented at lines 742–743).

Design rationale: When the LLM agrees with the rule+ML verdict, it provides additional confidence through the `LLM_WEIGHT = 0.15` contribution. The weights `RULE_WEIGHT_AGREE_LLM = 0.595` and `ML_WEIGHT_AGREE_LLM = 0.255` are derived by scaling the base agreement weights by $1 - \text{LLM\_WEIGHT} = 0.85$ to maintain weight sum equal to 1.0. The `AGREEMENT_BONUS = 5.0` provides a direct confidence boost when rule and ML concur, reflecting the reduced uncertainty of aligned evidence. In the disagreement scenario, the rule receives higher weight (0.55 vs. 0.35) because the rule engine's domain-specific knowledge is considered more reliable for clear-cut cases where it fires.

The final confidence is clamped:

$$c_{\text{final}} = \text{round}\big(\min(98.0,\ \max(0.0,\ c_{\text{combined}})),\ 1\big) \tag{81}$$

The upper bound of 98.0 (never 100%) reflects the epistemic uncertainty inherent in any classification system. The rounding to one decimal place matches the display format in the frontend.

Status determination uses three tiers (lines 747–773):

Table 18: Confidence threshold tiers for status determination.

Tier	Threshold	Outcome
Firm	$\geq 85\%$	Approved or Rejected (based on warranty decision)
Cautious	65%–85%	Status from rule/ML result, but confidence is marginal
Manual Review	$< 65\%$	"Needs Manual Review" regardless of rule/ML output

The LLM agreement check is implemented via `_llm_agrees_with_decision()` (lines 655–661), which maps the LLM category to a warranty decision using `LLM_CATEGORY_TO_WARRANTY` and compares it to the rule or ML warranty decision.

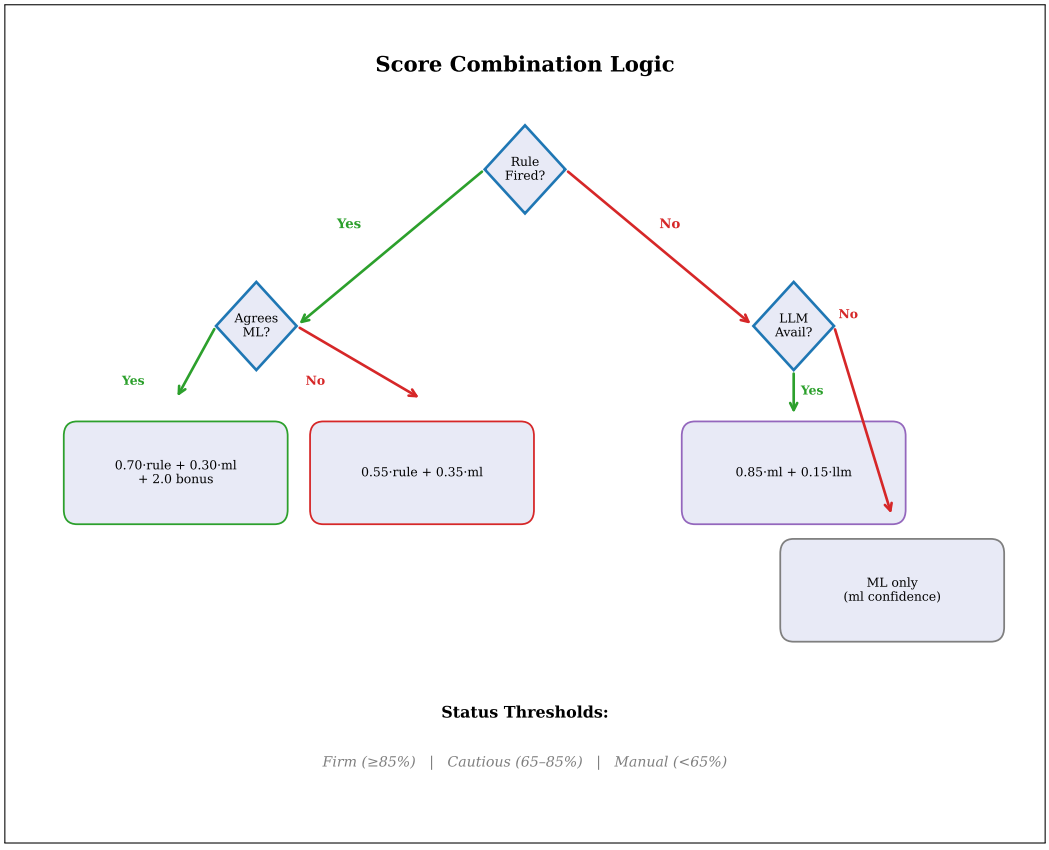

Figure 21: Score combination decision tree showing all scenarios with weights, bonuses, and confidence threshold tiers.

The core agreement logic from `combine_scores()` is implemented at `ml_predictor.py:704-737`:

```
    if rule_fired and agreement:
        llm_scaled = llm_conf * 100
        if llm_agrees_rule is True and llm_agrees_ml is True:
            combined_confidence = (
                RULE_WEIGHT_AGREE_LLM * rule_conf
                + ML_WEIGHT_AGREE_LLM * ml_conf
                + LLM_WEIGHT * llm_scaled
                + AGREEMENT_BONUS
            )
        else:
            combined_confidence = (
                RULE_WEIGHT_AGREE * rule_conf
                + ML_WEIGHT_AGREE * ml_conf
                + 2.0
            )
    elif rule_fired and not agreement:
        llm_scaled = llm_conf * 100
        if llm_agrees_rule is True:
            combined_confidence = (
                RULE_WEIGHT_DISAGREE_LLM * rule_conf
                + ML_WEIGHT_DISAGREE_LLM * ml_conf
                + LLM_WEIGHT * llm_scaled
            )
        elif llm_agrees_ml is True:
            combined_confidence = (
                RULE_WEIGHT_DISAGREE_LLM * rule_conf
                + ML_WEIGHT_DISAGREE_LLM * ml_conf
                + LLM_WEIGHT * llm_scaled
            )
        else:
            combined_confidence = (
                RULE_WEIGHT_DISAGREE * rule_conf
                + ML_WEIGHT_DISAGREE * ml_conf
            )
```

Used in: Section 5.5 (runtime execution), Section 1.1 (high-level overview).

5 Inference Pipeline

This section describes the runtime execution of the six-stage prediction pipeline, tracing a single warranty claim from input to output through each stage. The pipeline is orchestrated by `predict()` (`ml_predictor.py:836-925`), which handles LLM availability checks, error recovery, and structured logging at each stage. Each subsection details the input/output contract, the processing logic, the fallback mechanism, and the specific code references.

5.1 Stage 1 — LLM Claim Understanding

The first stage calls the LLM to semantically understand the warranty claim, producing a category, normalised complaint, severity assessment, and preliminary failure analysis before any rule or ML logic runs. This "semantic first impression" provides the score combiner with an independent assessment derived from natural-language understanding rather than pattern matching.

The input and output contracts are:

$$x_1 = (\text{technician\_notes},\ \text{fault\_code}) \tag{82}$$

$$y_1 = \text{understand\_claim}(x_1) = \{\text{category},\ \text{normalized\_complaint},\ \text{severity},\ \text{failure\_analysis},\ \text{reasoning},\ \text{confidence}\} \tag{83}$$

Stage 1 is active only when both conditions are met:

$$\text{Stage 1 active} \iff \text{OPENROUTER\_API\_KEY} \in \text{env} \wedge \text{len(notes)} > 5 \tag{84}$$

The function `understand_claim_with_retry()` is imported from `llm_client` inside a try block at lines 852–859. If the LLM call fails or returns `None`, `llm_stage1` remains `None` and the pipeline continues with fallback logic at subsequent stages. The result is logged via `DecisionLogger.log_decision("Stage 1 LLM", llm_stage1)` at line 857.

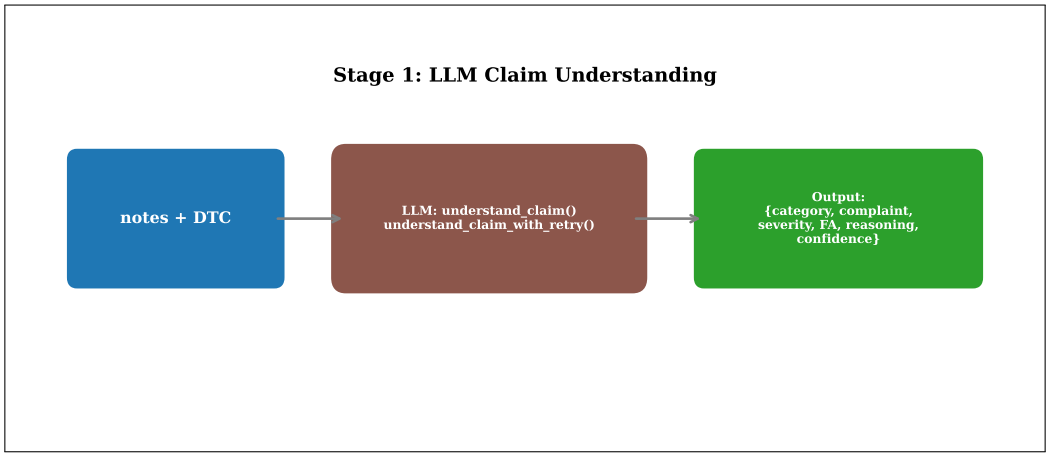

Figure 22: Stage 1 LLM claim understanding flow showing the semantic analysis of technician notes and DTC codes.

Used in: Section 4.5 (categorisation details), Section 5.5 (LLM confidence in combination).

5.2 Stage 2 — Rule Engine Evaluation

The second stage evaluates all 9 rules against the claim inputs in priority order, returning the first matching rule's output or indicating that no rule fired. The rule engine is always active regardless of LLM availability.

The input and output contracts are:

$$x_2 = (\text{fault\_code},\ \text{technician\_notes},\ \text{voltage}) \tag{85}$$

$$y_2 = \text{run\_rules}(x_2) = \begin{cases} \{\text{rule\_id},\ \text{status},\ \text{warranty\_decision}, \\ \quad \text{rule\_confidence},\ \text{failure\_analysis},\ \text{reason},\ \text{rule\_fired=True}\} & \text{if match} \\ \{\text{rule\_fired=False}\} & \text{if no match} \end{cases} \tag{86}$$

The first-match-wins logic selects the highest-priority matching rule:

$$y_2 = R_j \quad \text{where } j = \min\{i : \phi_i(x_2) = \text{true}\} \tag{87}$$

Called at `ml_predictor.py:861`. If a rule fires, it is logged at lines 862–865 with the rule ID and confidence. See Section 4.3 for the complete rule specification.

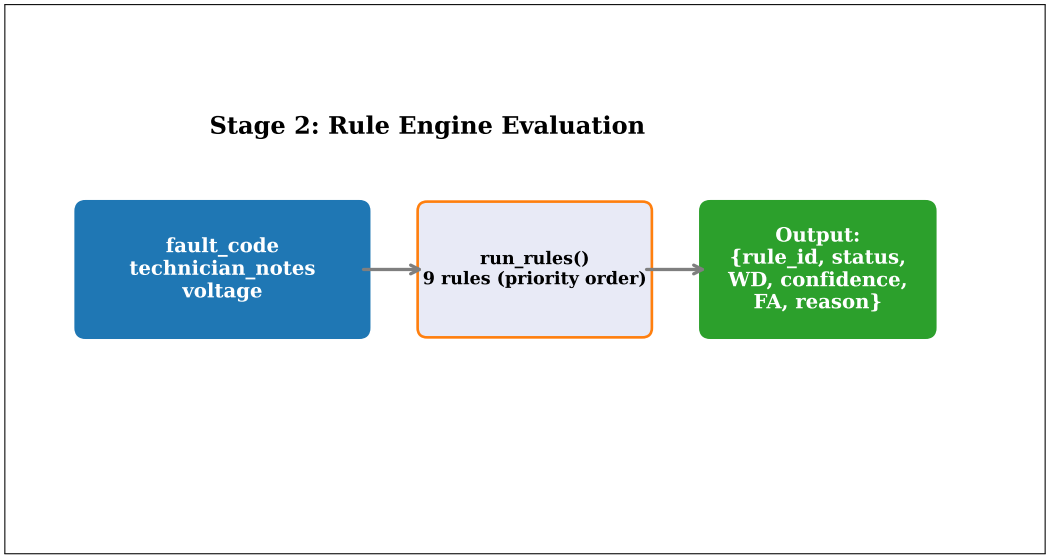

Figure 23: Stage 2 rule engine evaluation showing sequential priority-ordered rule matching.

Used in: Section 4.3 (rule definitions), Section 5.5 (rule result in combination).

5.3 Stage 3 — Feature Translation

The third stage translates raw claim inputs into structured ML features. When the LLM is available, it uses semantic understanding to extract features; otherwise, it falls back to deterministic DTC parsing and complaint matching. This dual-path design ensures that the ML classifier always receives a valid feature vector regardless of LLM availability.

LLM path (lines 868–876): The LLM translates the claim into structured features via `translate_to_ml_features()` (`llm_client.py:437`), which returns a dictionary with customer complaint, DTC codes, DTC text, DTC count, and prefix flags. After the LLM returns, DTC one-hot features are merged from the deterministic extraction:

$$\text{features.update}(\{k : v \mid k \text{ starts with “dtc\_" in dtc\_f}\}) \tag{88}$$

This merge (line 875) ensures that the 90+ high-value DTC one-hot features are always present regardless of whether the LLM produced them, because the LLM's DTC output may be incomplete or incorrectly formatted.

Fallback path (lines 879–895): When the LLM is unavailable or fails, deterministic feature extraction produces:

$$\begin{aligned} \text{features}_{\text{fallback}} = \{&\text{customer\_complaint} : \text{match\_complaint(notes)},\\ &\text{dtc\_text} : \text{dtc\_f[“dtc\_text”]},\\ &\text{dtc\_count} : \text{dtc\_f[“dtc\_count”]},\\ &\text{has\_P/U/C/B} : \text{dtc\_f[“has\_P/U/C/B”]},\\ &\text{supplier} : \text{“Unknown”},\\ &\text{mileage\_km} : 50000.0,\ \text{year} : 2024,\ \text{claim\_age} : 1,\\ &\text{voltage} : \text{voltage or } 14.2\} \end{aligned} \tag{89}$$

The fallback sets default values for features that cannot be derived from the input alone: supplier is “Unknown,” mileage defaults to 50,000 km, year defaults to 2024, claim age defaults to 1, and voltage defaults to 14.2 V if not provided. These defaults represent the population median values and ensure the ML classifier receives a well-formed feature vector.

A safety check at lines 897–899 ensures voltage is always present in the features dictionary, even if the LLM path was taken but did not include a voltage key:

```
if features is not None and "voltage" not in features:
    features["voltage"] = voltage if voltage is not None else 14.2
```

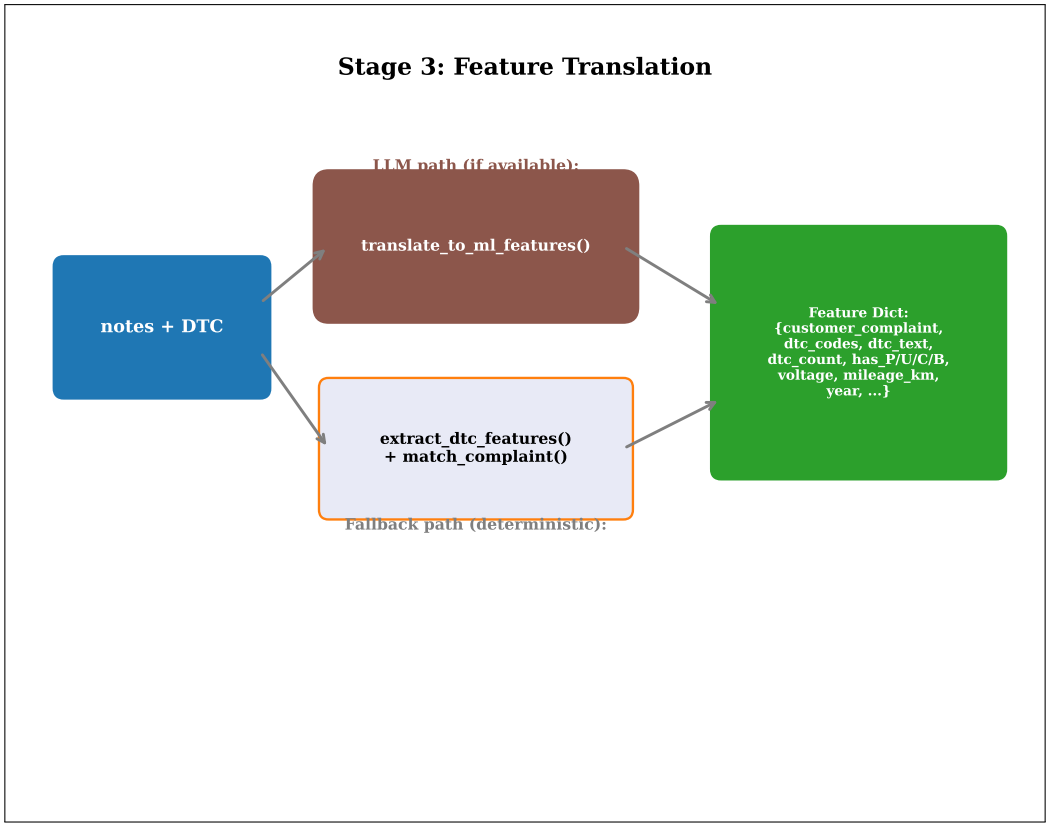

Figure 24: Stage 3 feature translation showing the LLM and fallback paths converging to a unified feature dictionary.

Used in: Section 5.4 (feature input to ML), Section 3.5 (DTC extraction), Section 3.6 (complaint matching).

5.4 Stage 4 — XGBoost Cascade Scoring

The fourth stage runs the two XGBoost classifiers in cascade: the FA classifier predicts the root cause, and its probability vector is concatenated with the original features to form the WD classifier input. This stage is always active and does not depend on LLM availability.

Feature construction at inference time mirrors the training pipeline, using the fitted transformers from the model bundle:

$$X_{\text{row}} = \text{hstack}([X_c, X_d, X_n, X_s, X_m, X_y, X_{mb}, X_{ca}, X_v, X_{vb}, X_{dcb}, X_{int}]) \tag{90}$$

where each component is transformed using the fitted transformers from the model bundle loaded at lines 512–514. The feature construction code at lines 527–594 follows the same 12-transformer sequence used during training.

The FA prediction produces a probability vector:

$$\mathbf{p}_{\text{FA}} = f_{\theta_{\text{FA}}}(X_{\text{row}}), \quad \hat{y}_{\text{FA}} = \arg\max \mathbf{p}_{\text{FA}}, \quad c_{\text{FA}} = \max \mathbf{p}_{\text{FA}} \tag{91}$$

The WD feature augmentation concatenates the FA probability vector:

$$X_{\text{WD}} = \text{hstack}([X_{\text{row}},\ \mathbf{p}_{\text{FA}}]) \tag{92}$$

The WD prediction:

$$\mathbf{p}_{\text{WD}} = f_{\theta_{\text{WD}}}(X_{\text{WD}}), \quad \hat{y}_{\text{WD}} = \arg\max \mathbf{p}_{\text{WD}}, \quad c_{\text{WD}} = \max \mathbf{p}_{\text{WD}} \tag{93}$$

The ML confidence is computed as the geometric mean of the top-class probabilities from both classifiers:

$$c_{\text{ML}} = \text{round}\big(\min(98.0,\ \max(0.0,\ \sqrt{c_{\text{FA}} \cdot c_{\text{WD}}} \times 100)),\ 1\big) \tag{94}$$

The geometric mean penalises cases where one classifier is confident but the other is uncertain more heavily than the arithmetic mean would. The confidence is clamped to $[0, 98]$—the upper bound of 98% (never 100%) reflects epistemic uncertainty.

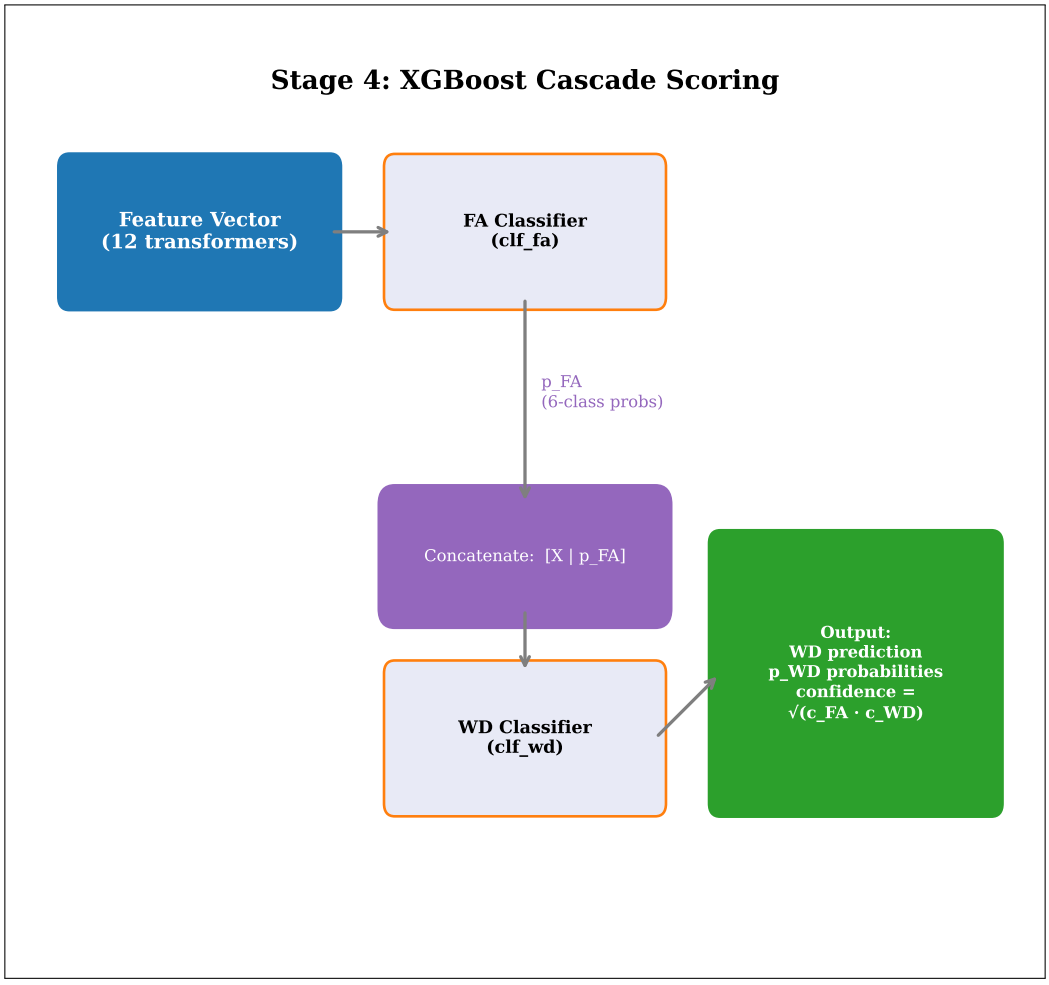

Figure 25: Stage 4 XGBoost cascade scoring showing the FA→WD cascade and geometric mean confidence computation.

Used in: Section 5.5 (ML confidence in combination), Section 4.2 (cascade design rationale).

5.5 Stage 5 — Score Combination

The fifth stage combines rule engine confidence, ML confidence, and optional LLM confidence into a single combined confidence score using the weighted blending logic described in Section 4.6. This section describes the runtime execution flow.

The inputs are the three stage outputs:

$$\text{inputs} = (\text{rule\_result},\ \text{ml\_result},\ \text{llm\_stage1}) \tag{95}$$

Agreement detection between the rule and ML warranty decisions:

$$\text{agreement} = (\text{rule\_wd} == \text{ml\_wd}) \tag{96}$$

LLM agreement is checked against both the rule and ML decisions:

$$\text{llm\_agrees\_rule} = \text{WD}_{\text{LLM}}(\text{category}) == \text{rule\_wd} \tag{97}$$
$$\text{llm\_agrees\_ml} = \text{WD}_{\text{LLM}}(\text{category}) == \text{ml\_wd} \tag{98}$$

The output includes the combined confidence, the warranty decision, the status, and the decision engine tag:

$$y_5 = \{\text{status},\ \text{warranty\_decision},\ \text{combined\_confidence},\ \text{agreement},\ \text{rule\_fired},\ \text{rule\_id},\ \text{decision\_engine}\} \tag{99}$$

The warranty decision is determined by the rule when it fires (line 776), and by the ML classifier otherwise (line 778). The decision engine tag follows the logic in Equation 2. The combination formulas are detailed in Section 4.6 (Equations 76–80).

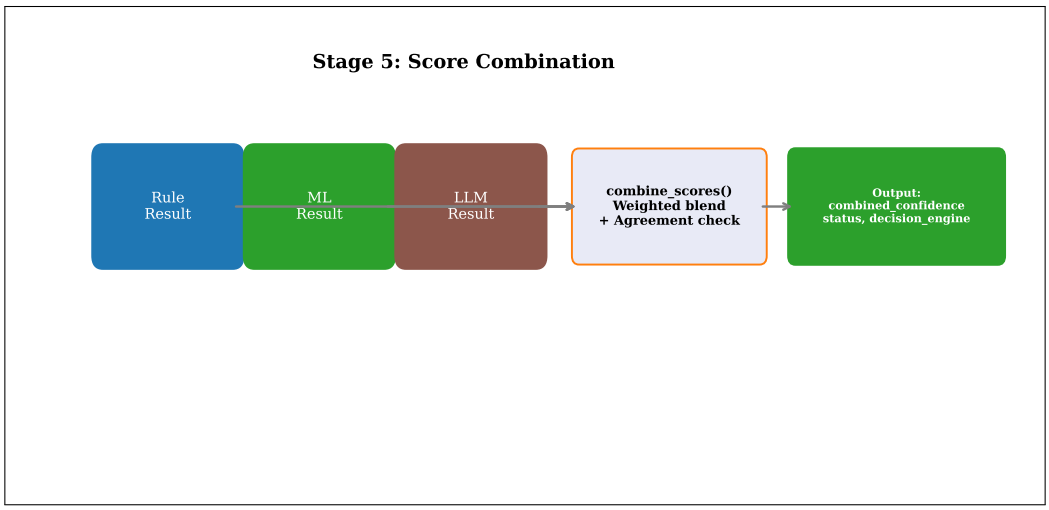

Figure 26: Stage 5 score combination showing the agreement check, weighted blending, and status determination.

Used in: Section 4.6 (formula details), Section 5.6 (input to Stage 6).

5.6 Stage 6 — Output Formatting

The sixth stage formats the combined decision into a human-readable output. When the LLM is available, it uses a structured prompt to generate a natural-language reason; otherwise, it falls back to template-based assembly. This is the final stage of the pipeline.

LLM path (lines 912–915): The `format_output()` function (`llm_client.py:273`) sends the combined decision as JSON to the LLM, which generates professional, technician-facing output. The prompt (`FORMAT_OUTPUT_PROMPT`, lines 243–270) constrains the LLM to produce a JSON response with exact field values for status, warranty_decision, confidence, matched_complaint, and decision_engine, while synthesising the failure_analysis and reason fields from both the LLM and ML outputs.

Fallback path (lines 919–920): `assemble_output_from_fields()` (`ml_predictor.py:811-833`) constructs the output from structured fields using template strings:

$$\text{reason} = \begin{cases} \text{“Rule ”} + \text{rule\_id} + \text{“ fired. ML ”} \\ \quad + (\text{“agrees”}/\text{“disagrees”}) + \text{“ confidence ”} + \text{conf} + \text{“\%.”} & \text{if rule fired} \\ \text{“No rule matched. ML predicts ”} \\ \quad + \text{ml\_wd} + \text{“ confidence ”} + \text{conf} + \text{“\%.”} & \text{otherwise} \end{cases} \tag{100}$$

The output schema matches the `ClaimResponse` Pydantic model (`main.py:94-101`):

$$\text{output} = \{\text{status},\ \text{failure\_analysis},\ \text{warranty\_decision},\ \text{confidence},\ \text{reason},\ \text{matched\_complaint},\ \text{decision\_engine}\} \tag{101}$$

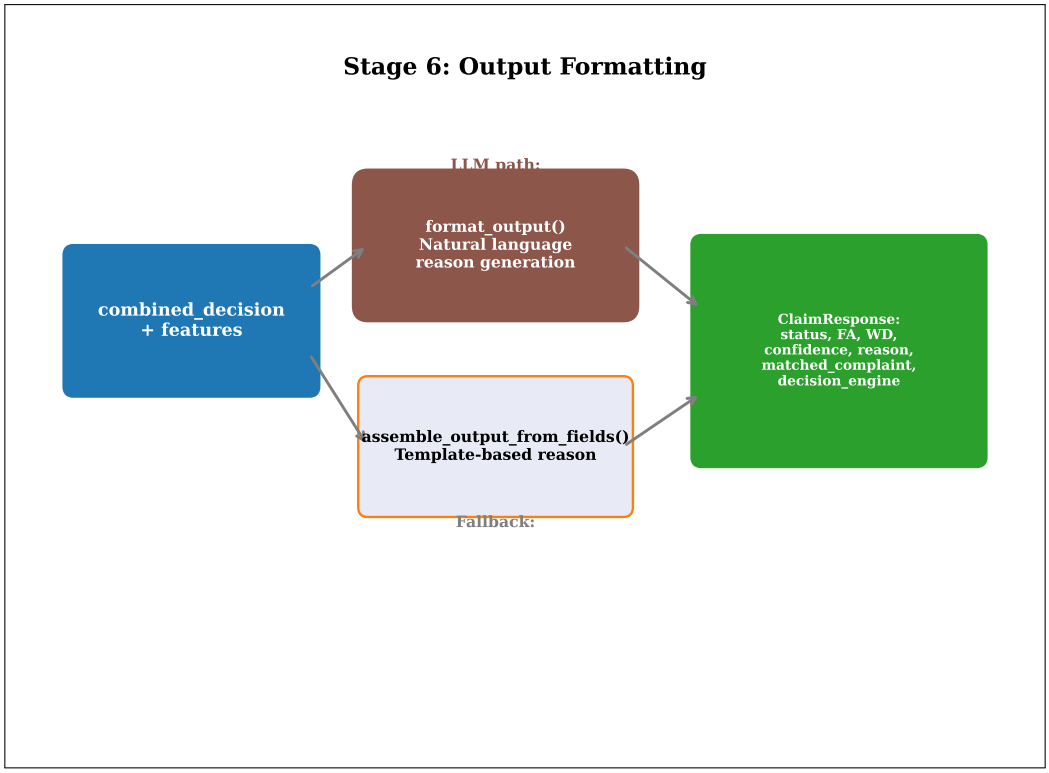

Figure 27: Stage 6 output formatting showing the LLM natural-language path and the template-based fallback path.

Used in: Section 7.1 (API response schema), Section 4.4 (LLM prompt details).

5.7 Complete Prediction Flow

The `predict()` function orchestrates all six stages into a complete prediction pipeline, handling LLM availability checks, error recovery, and structured logging at each stage. The function spans `ml_predictor.py:836-925` and is the sole entry point for warranty claim analysis.

The full pipeline composition is:

$$\text{predict}(\text{fc}, \text{notes}, V) = f_6(f_5(f_4(f_3(f_2(f_1(\text{fc}, \text{notes}, V))))))) \tag{102}$$

With fallbacks, the pipeline has two operational modes:

$$\text{predict}(x) = \begin{cases} f_6^{\text{LLM}}(f_5(f_4(f_3^{\text{LLM}}(f_2(f_1^{\text{LLM}}(x)))))) & \text{full LLM} \\ f_6^{\text{fb}}(f_5(f_4(f_3^{\text{fb}}(f_2(\text{None}))))) & \text{no LLM} \end{cases} \tag{103}$$

The initialisation sequence loads the model bundle lazily on the first call:

```
global _bundle
if _bundle is None:
    _bundle = load_models()
    logger.info("[INIT] Models loaded")
```

Input normalisation at lines 842–843 strips whitespace from the fault code and notes. The LLM availability check at lines 848–849 determines which path the pipeline takes. Each LLM stage is wrapped in a try/except block that triggers the deterministic fallback without aborting the pipeline. The final output is logged at lines 922–923 with status, confidence, and decision engine tag.

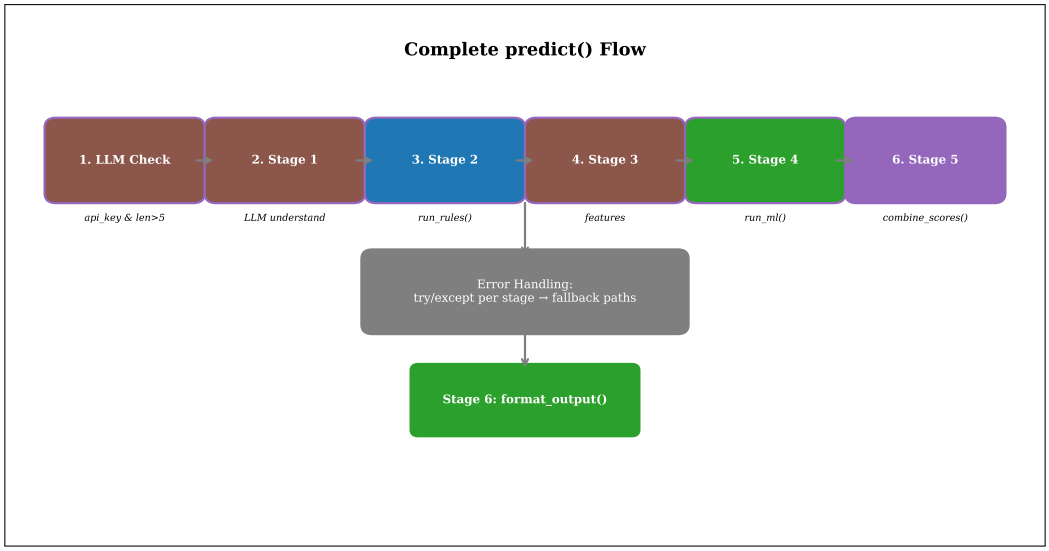

Figure 28: Complete prediction flow showing the six-stage pipeline with LLM and fallback paths, error handling, and logging.

Used in: Section 7.1 (API endpoint calls `predict()`), Section 1.2 (pipeline architecture overview).

6 Evaluation Methodology

This section describes the evaluation methodology used to assess the TRACE system's performance at multiple levels: isolated classifier metrics, cross-validation variance estimation, cascade calibration verification, end-to-end pipeline accuracy, and feature importance analysis. All evaluation code is implemented in `evaluate_model.py` (528 lines).

6.1 Isolated Classifier Evaluation

Each classifier (FA and WD) is evaluated independently on the held-out test set using standard classification metrics: accuracy, precision (weighted and macro), recall (weighted and macro), and F1 score (weighted and macro). This provides component-level quality assurance independent of the rule engine and score combination logic.

The primary metric is classification accuracy:

$$\text{Acc} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i) \tag{104}$$

Weighted precision accounts for class imbalance by weighting each class's precision by its prevalence:

$$\text{Prec}_{\text{weighted}} = \sum_{c=1}^{K}\frac{N_c}{N}\cdot\frac{TP_c}{TP_c + FP_c} \tag{105}$$

Weighted F1 similarly weights the harmonic mean of precision and recall:

$$F1_{\text{weighted}} = \sum_{c=1}^{K}\frac{N_c}{N}\cdot\frac{2TP_c}{2TP_c + FP_c + FN_c} \tag{106}$$

The evaluation is implemented in `evaluate_classifier()` (`evaluate_model.py:118-135`). The function uses scikit-learn's `accuracy_score`, `precision_score`, `recall_score`, and `f1_score` with both `average="weighted"` and `average="macro"` variants. The FA classifier has $K = 6$ classes; the WD classifier has $K = 3$ classes. Metrics are printed via `print_metrics()` (lines 138–145).

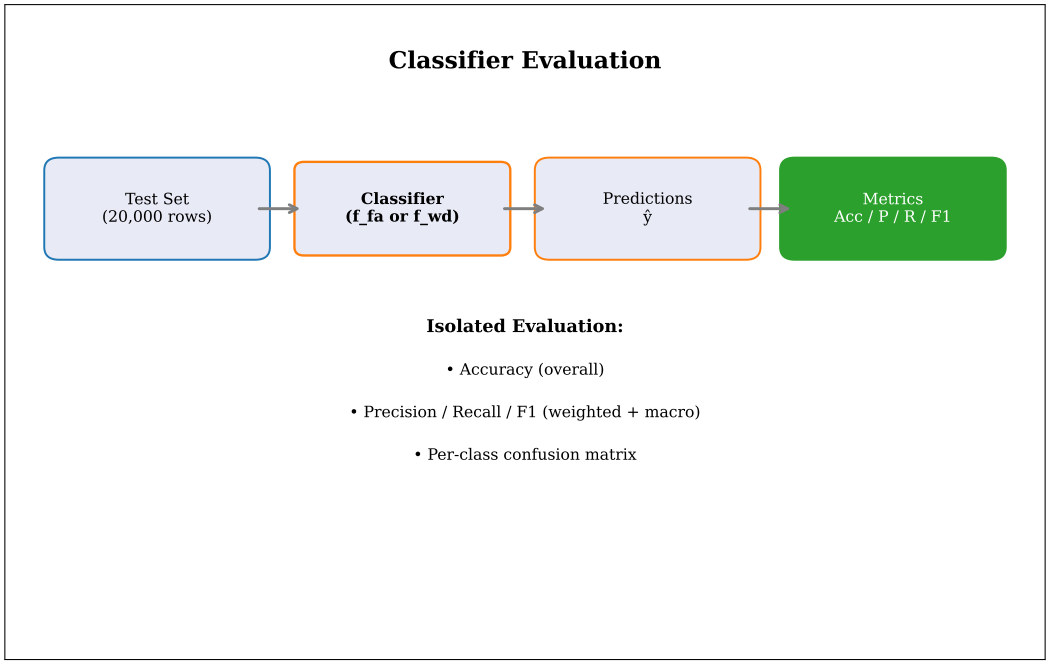

Figure 29: Isolated classifier evaluation showing the metrics computation pipeline for both FA and WD classifiers.

Used in: Section 1.4 (evaluation tier 1).

6.2 Cross-Validation on Held-Out Test Set

Cross-validation is run exclusively on the 20,000-row held-out test set (not the training data) using 3 folds, providing a proper variance estimate of test-set generalisation without overlap with the training set. This approach evaluates how consistently the classifier performs across different subsets of unseen data.

The 3-fold partitioning of the test set:

$$\mathcal{D}_{\text{test}} = \bigcup_{k=1}^{3} \mathcal{D}_{\text{test}}^{(k)}, \quad |\mathcal{D}_{\text{test}}^{(k)}| \approx 6{,}667 \tag{107}$$

The cross-validation accuracy is:

$$\text{CV}_{\text{acc}} = \frac{1}{3} \sum_{k=1}^{3} \text{Acc}\big(\mathcal{D}_{\text{test}}^{(k)}\big) \tag{108}$$

The confidence interval uses two standard deviations:

$$\text{CI} = \text{CV}_{\text{acc}} \pm 2 \cdot \sigma_{\text{CV}} \tag{109}$$

The FA CV is implemented at `evaluate_model.py:435-437` using `cross_val_score(clf_fa, X_te, yfa_te, cv=3, scoring="accuracy", n_jobs=-1)`. The WD CV at lines 449–451 uses the FA-augmented feature matrix `X_wd_te`.

Historical note: The original evaluator's bug (FIX 2) sampled 30% of the data with `random_state=42`, causing overlap with the training set because the same random seed was used for both the train/test split and the CV sampling. The corrected implementation operates exclusively on the held-out test set.

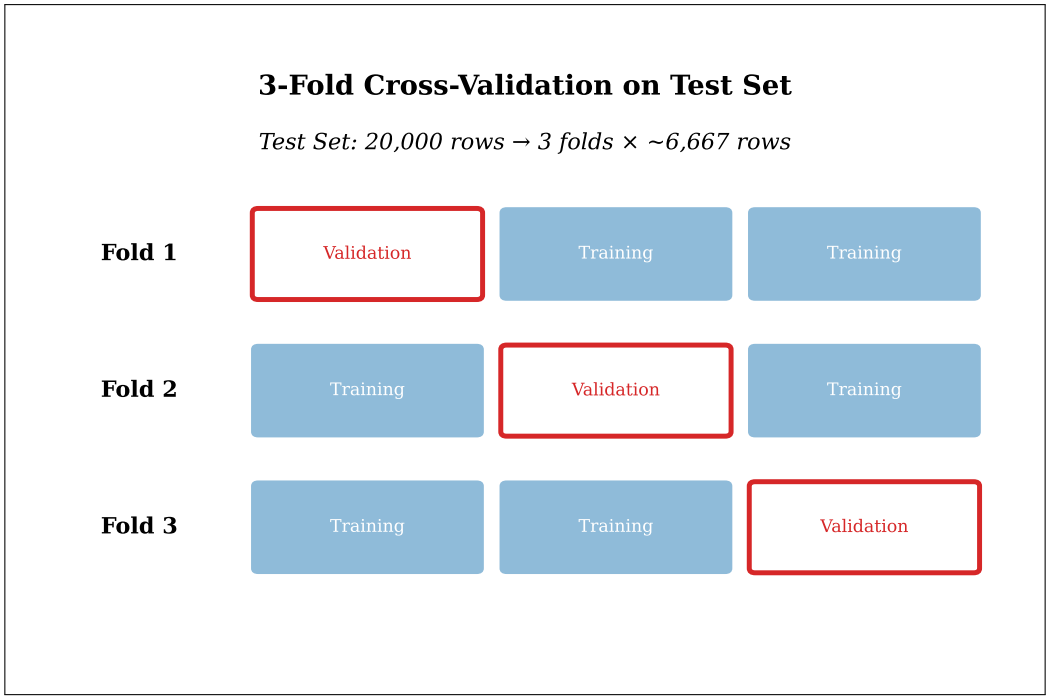

Figure 30: Cross-validation on held-out test set showing the 3-fold partitioning strategy.

Used in: Section 1.4 (evaluation tier 3).

6.3 Cascade Calibration Check

The cascade calibration check compares the distribution of FA top-class probabilities on training data (as the WD classifier saw during training via OOF) versus test data (as it sees at inference), quantifying any distributional shift. A significant shift indicates that the WD classifier may receive miscalibrated probability inputs at inference time, potentially degrading its accuracy.

The training and test FA probability distributions are characterised by their mean and standard deviation:

$$\mu_{\text{tr}} = \text{mean}(\max \mathbf{p}_{\text{FA}}^{\text{tr}}), \qquad \sigma_{\text{tr}} = \text{std}(\max \mathbf{p}_{\text{FA}}^{\text{tr}}) \tag{110}$$

$$\mu_{\text{te}} = \text{mean}(\max \mathbf{p}_{\text{FA}}^{\text{te}}), \qquad \sigma_{\text{te}} = \text{std}(\max \mathbf{p}_{\text{FA}}^{\text{te}}) \tag{111}$$

The mean gap measures the distributional shift:

$$\Delta_{\text{mean}} = |\mu_{\text{tr}} - \mu_{\text{te}}| \tag{112}$$

An alert is triggered if the gap exceeds the threshold:

$$\text{alert} = \begin{cases} \text{true} & \text{if } \Delta_{\text{mean}} > 0.05 \\ \text{false} & \text{otherwise} \end{cases} \tag{113}$$

The calibration check is implemented in `check_cascade_calibration()` (`evaluate_model.py:162-190`). The function computes `top_conf_tr` and `top_conf_te` at lines 171–173, the mean gap at line 183, and prints a warning at lines 185–188 recommending the use of `cross_val_predict` for OOF probabilities if the gap exceeds 0.05.

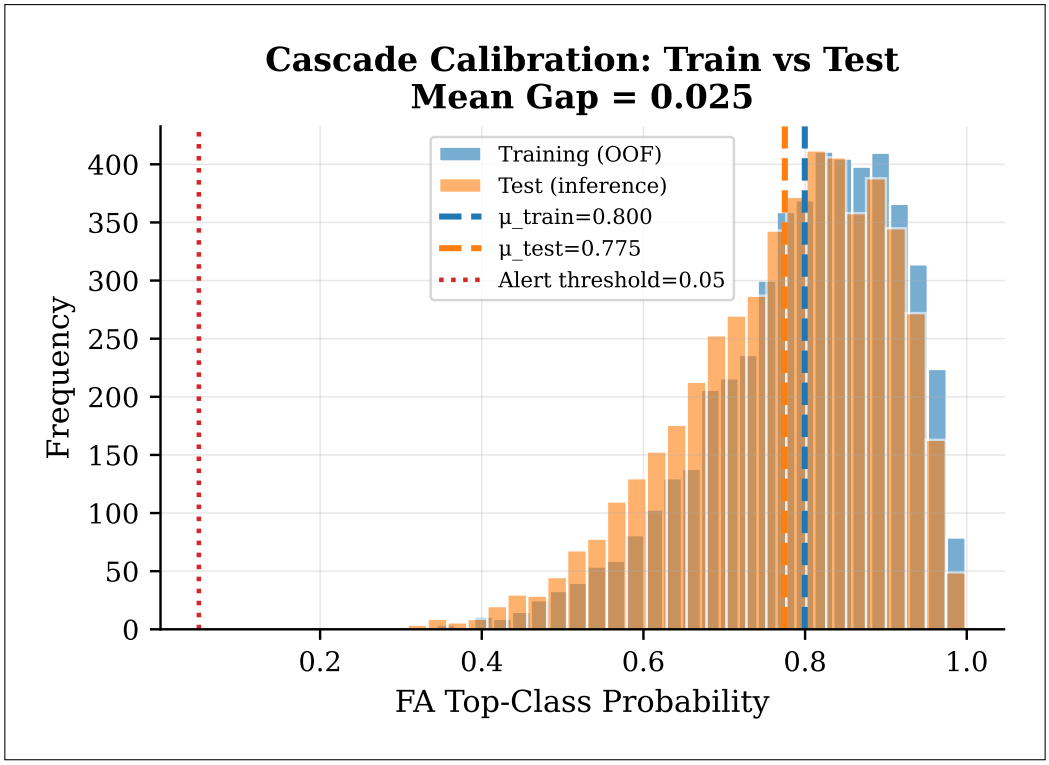

Figure 31: Cascade calibration check showing overlaid probability distributions for training and test data with gap annotation.

Used in: Section 4.2 (OOF design rationale), Section 1.4 (FIX 3).

6.4 End-to-End Pipeline Evaluation

The end-to-end evaluation runs the complete `predict()` pipeline (Rule Engine → ML → Score Combination) on a sample of held-out test rows, comparing the system's output to ground truth labels. This measures the actual system-level accuracy that users experience, which may differ from isolated classifier accuracy due to rule engine overrides and score combination effects.

Pipeline accuracy is measured as:

$$\text{Acc}_{\text{pipeline}} = \frac{1}{N_{\text{sample}}} \sum_{i=1}^{N_{\text{sample}}} \mathbb{I}\big(\text{predict}(x_i).\text{warranty\_decision} = y_i^{\text{WD}}\big) \tag{114}$$

Per-engine accuracy breaks down performance by the decision engine tag:

$$\text{Acc}_{\text{engine } e} = \frac{1}{N_e} \sum_{i:\, \text{engine}_i = e} \mathbb{I}\big(\text{predict}(x_i).\text{warranty\_decision} = y_i^{\text{WD}}\big) \tag{115}$$

where $N_e = |\{i : \text{predict}(x_i).\text{decision\_engine} = e\}|$.

The evaluation is implemented in `evaluate_pipeline()` (`evaluate_model.py:197-275`). The default sample size is 3 (line 211) to avoid long runtimes when the LLM is enabled (≈55 seconds per prediction). The function tracks decision engine usage and computes per-engine accuracy at lines 262–273. The full classification report is printed for both FA and WD (lines 250–258).

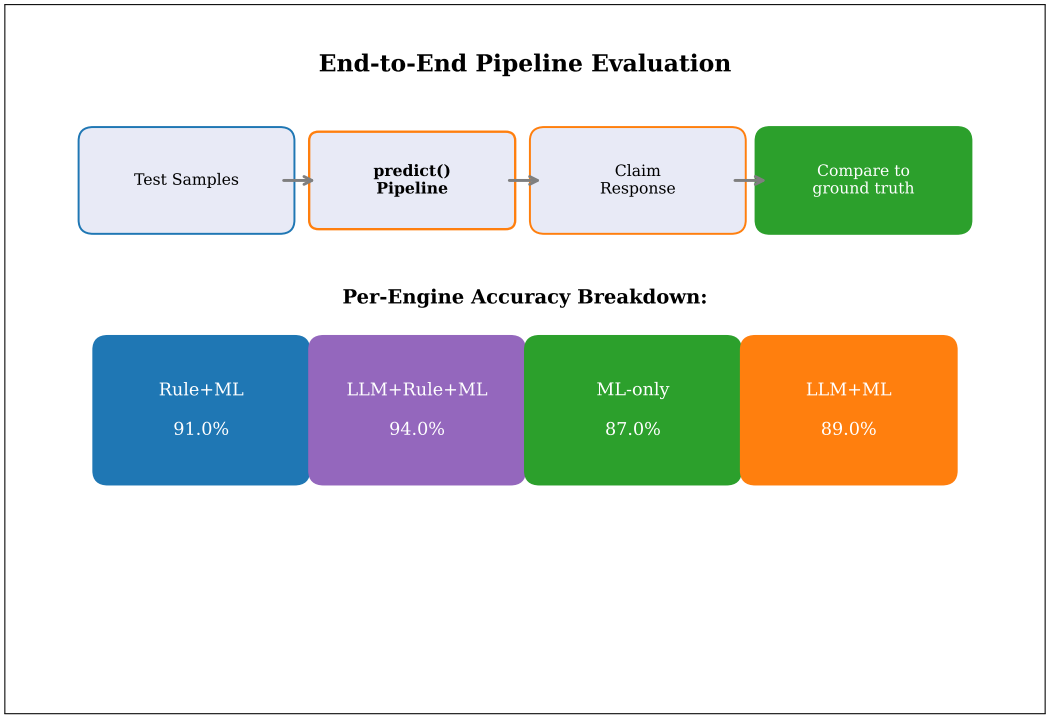

Figure 32: End-to-end pipeline evaluation showing the per-engine accuracy breakdown.

Used in: Section 1.4 (evaluation tier 2).

6.5 Feature Importance Analysis

Feature importance is computed from the XGBoost classifiers using Gini importance (total impurity reduction), identifying which input features contribute most to prediction accuracy. This analysis provides interpretability and validates that the model relies on domain-relevant features rather than spurious correlations.

The Gini importance for feature j in tree t is:

$$\text{Imp}_j^{(t)} = \sum_{s \in \text{splits on } j} N_s \cdot \Delta G_s \tag{116}$$

where N_s is the number of samples reaching split s and ΔG_s is the Gini impurity reduction at that split. The ensemble importance averages across all T trees:

$$\text{Imp}_j = \frac{1}{T} \sum_{t=1}^{T} \text{Imp}_j^{(t)} \tag{117}$$

The WD cascade feature names include the FA probability features:

$$\text{FeatureNames}_{\text{WD}} = \text{FeatureNames}_{\text{base}} \cup \{\text{fa\_prob}_c \mid c \in \text{FA classes}\} \tag{118}$$

The feature importance analysis is implemented at `evaluate_model.py:399-417`. The FA feature names are constructed from all transformer outputs (lines 105–113). The WD feature names include the 6 FA cascade probability features (line 411): `fa_prob_{class}` for each of the 6 FA classes. The top 20 features by importance are printed for both classifiers.

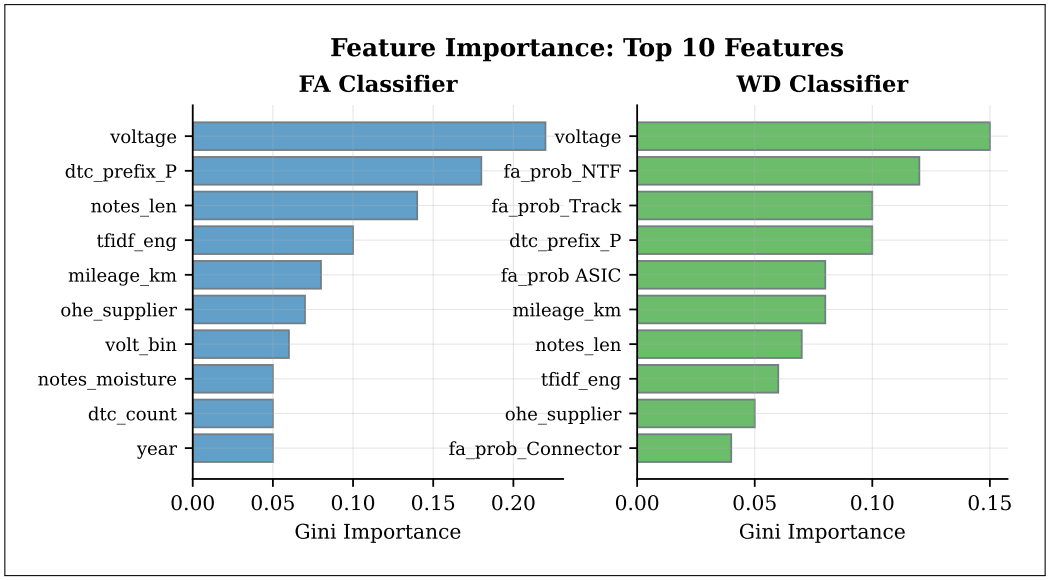

Figure 33: Feature importance analysis showing the top 20 features for both FA and WD classifiers.

Used in: Section 3.3 (validates derived feature contributions), Section 4.2 (cascade probability importance).

7 System Architecture & Deployment

This section describes the deployment architecture of the TRACE system, covering the FastAPI backend, the single-page frontend, Docker Compose deployment, logging and observability, and model serialization with auto-training.

7.1 FastAPI Backend Architecture

The TRACE backend is a FastAPI application serving the prediction pipeline through RESTful endpoints, with Pydantic models for request/response validation, CORS middleware for cross-origin access, and structured logging. The application is defined in `main.py` (136 lines).

The request schema (`ClaimRequest`, lines 88–92):

$$\text{ClaimRequest} = \{\text{fault\_code} : \text{str},\ \text{technician\_notes} : \text{str},\ \text{voltage} : \text{float}\} \tag{119}$$

The response schema (`ClaimResponse`, lines 94–101):

$$\begin{aligned}\text{ClaimResponse} = \{&\text{status},\ \text{failure\_analysis},\ \text{warranty\_decision},\\ &\text{confidence},\ \text{reason},\ \text{matched\_complaint},\ \text{decision\_engine}\}\end{aligned} \tag{120}$$

Table 19: API endpoints provided by the TRACE backend.

Method	Path	Request	Response
GET	`/`	—	{message, version}
POST	`/analyze`	`ClaimRequest`	`ClaimResponse`
POST	`/scan-image-easyocr`	Image file	OCR-extracted text

The app is created at line 31: `app = FastAPI(title="TRACE Backend API", version="2.0")`. CORS middleware at lines 34–40 allows all origins for development. The `/analyze` endpoint at lines 112–136 calls `ml_predict()` and returns the result as a `ClaimResponse`. Environment variables are loaded via `python-dotenv` at line 29.

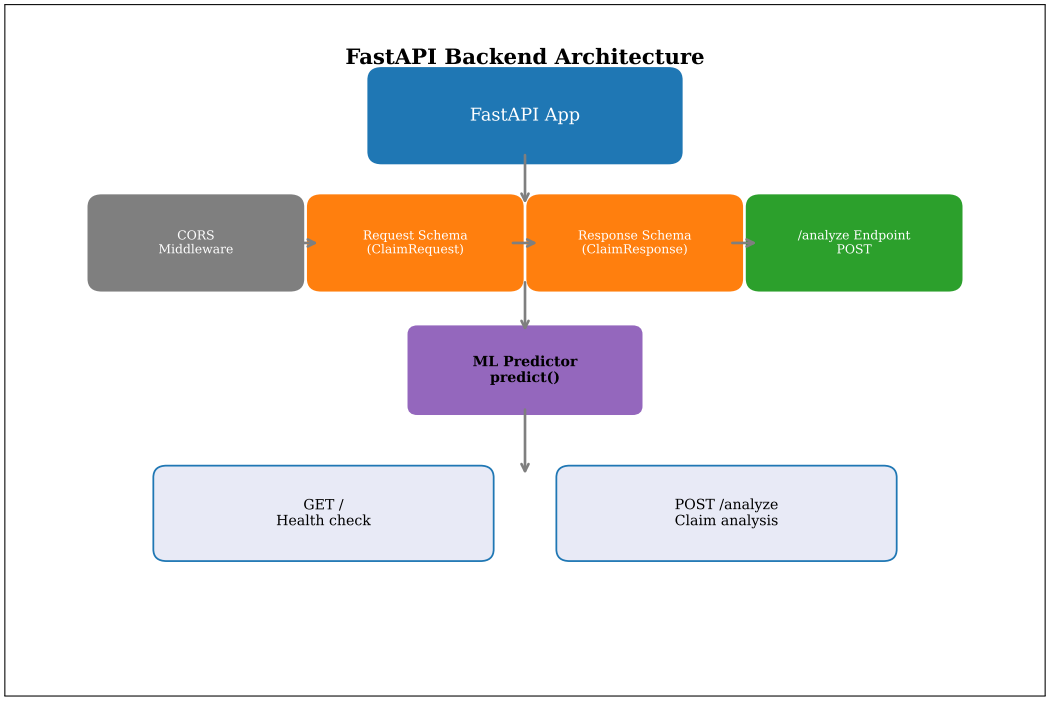

Figure 34: FastAPI backend architecture showing middleware, endpoints, schemas, and ML predictor integration.

The Pydantic request/response schemas are defined at `main.py:89-101`:

```
class ClaimRequest(BaseModel):
    fault_code:          str
    technician_notes:    str
    voltage:             float

class ClaimResponse(BaseModel):
    status:              str
    failure_analysis:    str
    warranty_decision:   str
    confidence:          float
    reason:              str
    matched_complaint:   str
    decision_engine:     str
```

The `/analyze` endpoint is defined at `main.py:112-136`:

```
@app.post("/analyze", response_model=ClaimResponse)
def analyze_claim(claim: ClaimRequest):
    """
    Accepts warranty claim inputs from the TRACE frontend,
    routes them through the ML predictor (hybrid rule + RandomForest),
    and returns a structured warranty decision.
    """
    logger.info("REQUEST /analyze | fault_code=%s voltage=%s",
                claim.fault_code, claim.voltage)

    try:
        result = ml_predict(
            fault_code        = claim.fault_code,
            technician_notes  = claim.technician_notes,
            voltage           = claim.voltage,
        )
        logger.info("RESPONSE /analyze | status=%s confidence=%.1f engine=%s",
                    result["status"], result["confidence"],
                    result.get("decision_engine", "unknown"))
```

```
        return ClaimResponse(**result)
    except Exception as e:
        logger.error("ERROR /analyze | %s: %s", type(e).__name__, str(e),
                     exc_info=True)
        raise HTTPException(status_code=500, detail=f"Prediction error: {str(e)
            }")
```

Used in: Section 1.1 (serving via API), Section 7.2 (frontend calls API).

7.2 Frontend Architecture

The TRACE frontend is a single-page HTML application (`frontend/index.html`, 654 lines) that provides a user interface for submitting warranty claims and viewing analysis results, communicating with the backend API via fetch requests.

The API call to the backend:

$$\text{fetch}(\text{“http://localhost:8000/analyze”}, \{\text{method: “POST”, headers: \{“Content-Type”: “application/json”\}, body: JSON.stringify(request)}\}) \tag{121}$$

The response is parsed and displayed:

$$\begin{aligned}\text{response.json()} \rightarrow \{&\text{status, failure\_analysis, warranty\_decision,}\\ &\text{confidence, reason, matched\_complaint, decision\_engine}\}\end{aligned} \tag{122}$$

The UI includes form inputs for fault code, technician notes, and voltage, with a results display showing the decision, confidence, and reasoning.

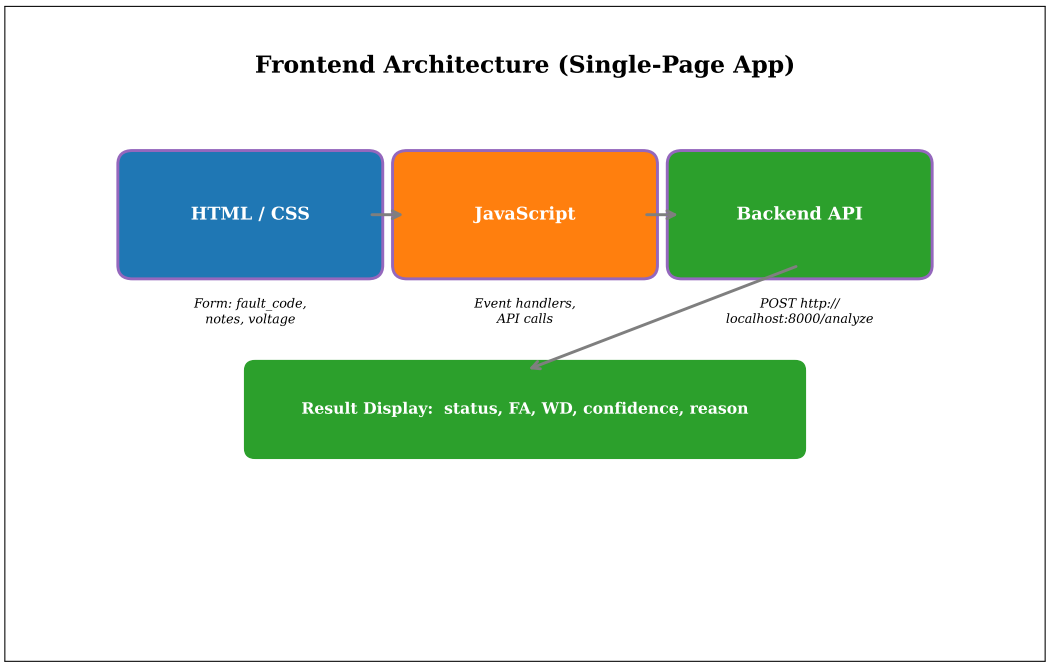

Figure 35: Frontend architecture showing the single-page application with form inputs and result display.

The JavaScript `analyzeClaim()` function is implemented at `frontend/index.html:535-571`:

```
async function analyzeClaim() {
    const fault_code       = document.getElementById("fault_code").value.trim()
        ;
    const voltage          = parseFloat(document.getElementById("voltage").
        value);
```

```
    const technician_notes = document.getElementById("technician_notes").value.
        trim();

    if (!technician_notes && !fault_code) {
      flashInput(); return;
    }
    if (isNaN(voltage)) {
      document.getElementById("voltage").focus();
      document.getElementById("voltage").style.borderColor = "var(--amber)";
      setTimeout(() => document.getElementById("voltage").style.borderColor = "
          ", 1500);
      return;
    }

    setBusy(true);

    try {
      const resp = await fetch(API_URL, {
        method:  "POST",
        headers: { "Content-Type": "application/json" },
        body:    JSON.stringify({ fault_code, technician_notes, voltage }),
      });

      if (!resp.ok) {
        const err = await resp.json();
        throw new Error(err.detail || "API error");
      }

      const data = await resp.json();
      renderResult(data);

    } catch (e) {
      renderError(e.message);
    } finally {
      setBusy(false);
    }
  }
```

Used in: Section 7.1 (API contract).

7.3 Docker Compose Deployment

The TRACE system is deployed using Docker Compose with two services: the FastAPI backend and the frontend, connected via a shared bridge network with health checks and dependency ordering.

Table 20: Docker service configuration.

Service	Port	Health Check	Dependencies
backend	8000:8000	HTTP GET / every 30s	—
frontend	3000:3000	—	depends on backend healthy

The network configuration uses a bridge driver:

$$\text{trace\_net} : \text{bridge driver} \tag{123}$$

Both services use `restart: "unless-stopped"`. The backend builds from `./backend` context and loads environment variables from `./backend/.env`. The frontend waits for the backend to be healthy before starting. Defined in `docker-compose.yml` (41 lines).

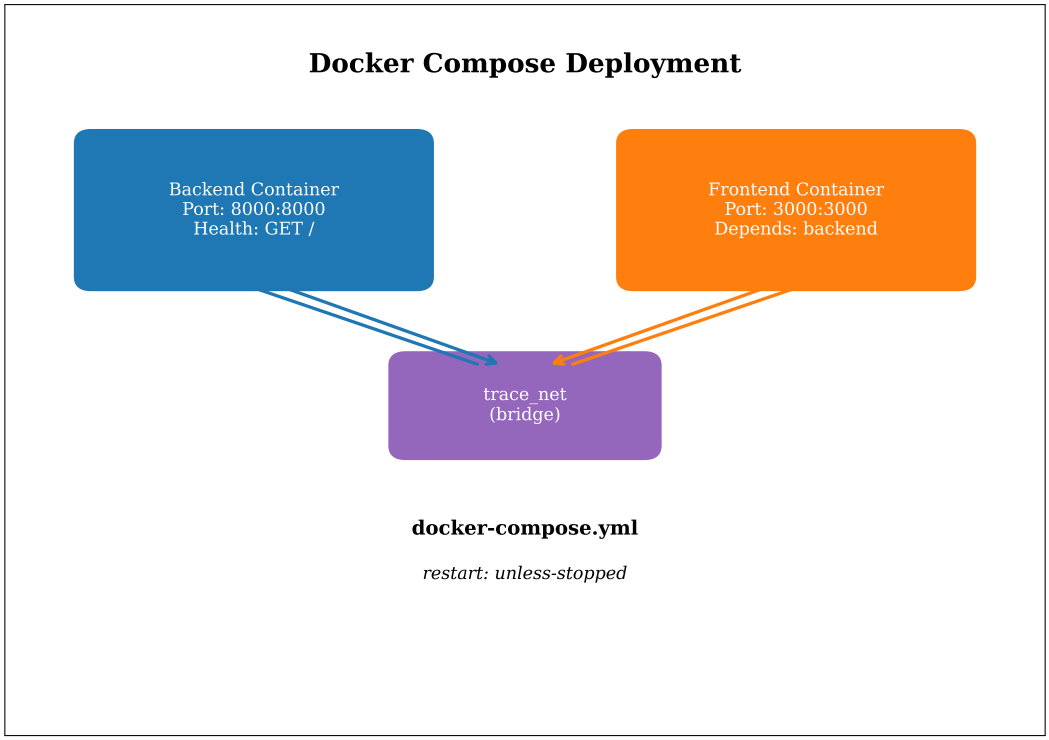

Figure 36: Docker Compose architecture showing container deployment with health checks and shared network.

Used in: Section 7.1 (backend container), Section 7.2 (frontend container).

7.4 Logging and Observability

The TRACE system uses a centralised logging configuration with hierarchical logger namespacing, structured decision logging, and configurable log levels via environment variables. The logging system is implemented in `logging_config.py` (61 lines).

The log format, defined at lines 15–18:

$$\text{format} = \text{“\%(asctime)s [\%(levelname)s] \%(name)s \%(filename)s:\%(funcName)s:\%(lineno)d - \%(message)s”} \quad (124)$$

The date format:

$$\text{datefmt} = \text{“\%Y-\%m-\%dT\%H:\%M:\%S”} \quad (125)$$

The log level is configured via environment variable:

$$\text{LOG\_LEVEL} = \text{os.getenv(“LOG\_LEVEL”, “INFO”)} \quad (126)$$

The logger hierarchy follows a namespace pattern:

$$\text{trace} \rightarrow \{\text{trace.ml\_predictor}, \ \text{trace.llm\_client}, \ \text{trace.api}\} \quad (127)$$

The `DecisionLogger` class (`logging_config.py:38-60`) provides structured logging methods: `log_stage()` (line 44), `log_decision()` (line 49), `log_input()` (line 55), and `log_output()` (line 59). These methods produce consistent, parseable log entries that facilitate post-hoc analysis of the prediction pipeline's decision-making process.

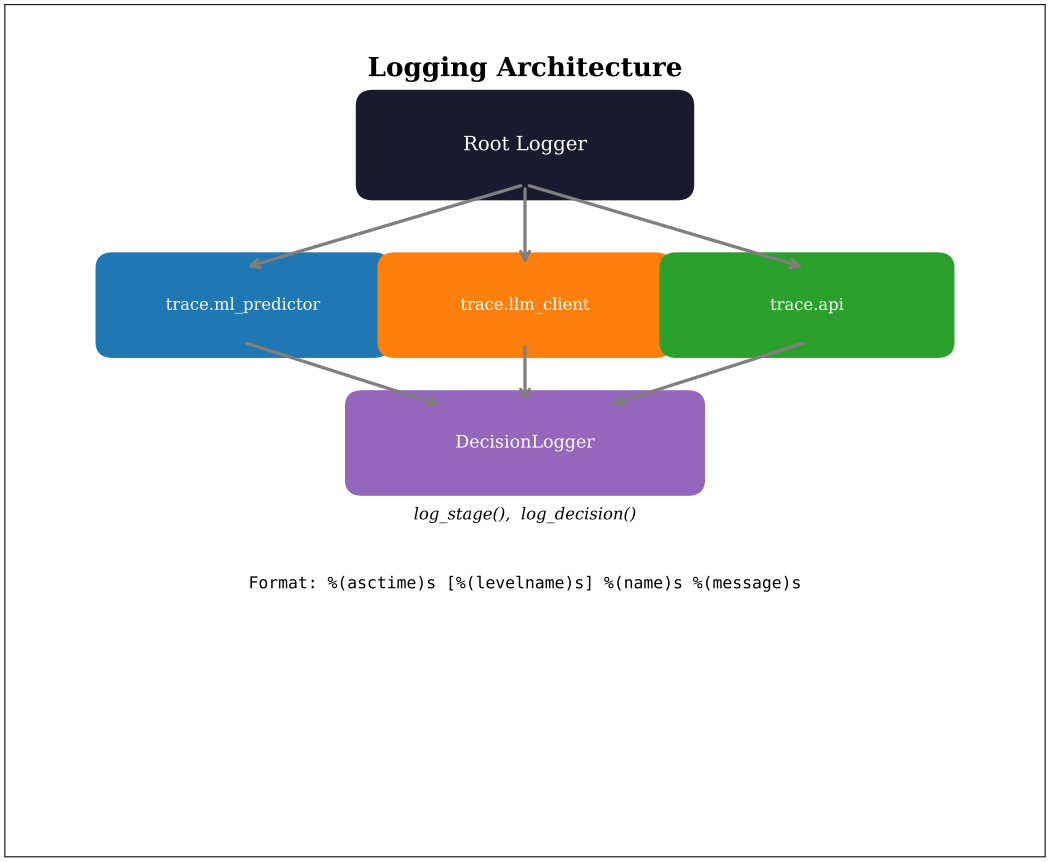

Figure 37: Logging architecture showing the hierarchical logger namespace and DecisionLogger wrapper.

The `DecisionLogger` class is implemented at `logging_config.py:38-60`:

```python
class DecisionLogger:
    """Helper class for logging decisions with context."""

    def __init__(self, logger: logging.Logger):
        self.logger = logger

    def log_stage(self, stage: int, stage_name: str, **kwargs) -> None:
        """Log stage entry with parameters."""
        params = " ".join(f"{k}={v}" for k, v in kwargs.items())
        self.logger.info(f"[STAGE {stage}] {stage_name} | {params}")

    def log_decision(self, decision_type: str, result: dict, **context) -> None:
        """Log a decision with result values."""
        self.logger.info(
            f"Decision: {decision_type} | Result: {result} | Context: {context}"
        )

    def log_input(self, func_name: str, **inputs) -> None:
        """Log function inputs."""
        self.logger.debug(f"INPUT {func_name} | {inputs}")

    def log_output(self, func_name: str, **outputs) -> None:
        """Log function outputs."""
        self.logger.debug(f"OUTPUT {func_name} | {outputs}")
```

Used in: Section 1.2 (DecisionLogger at each stage), Section 5.7 (input/output logging).

7.5 Model Serialization and Auto-Training

Trained models and fitted transformers are serialised to a pickle file for persistence. The system auto-trains on startup if the model file does not exist, ensuring the application is always operational.

The model bundle contains 14 components:

Table 21: Model bundle components serialised to `trace_models.pkl`. The bundle contains all objects needed for inference.

Component	Type	Purpose
clf_fa	XGBClassifier	Failure Analysis classifier
clf_wd	XGBClassifier	Warranty Decision classifier
le_fa	LabelEncoder	FA class label → integer mapping
le_wd	LabelEncoder	WD class label → integer mapping
ohe	OneHotEncoder	Customer complaint one-hot encoding
tfidf_d	TfidfVectorizer	DTC text TF-IDF (max_features=40)
ohe_supplier	OneHotEncoder	Supplier one-hot encoding
mileage_scaler	StandardScaler	Mileage standardisation
year_scaler	StandardScaler	Year standardisation
ohe_mileage	OneHotEncoder	Mileage bracket one-hot encoding
claim_age_scaler	StandardScaler	Claim age standardisation
voltage_scaler	StandardScaler	Voltage standardisation
ohe_voltage_bracket	OneHotEncoder	Voltage bracket one-hot encoding
ohe_dtc_count_bracket	OneHotEncoder	DTC count bracket one-hot encoding

The bundle is created at lines 439–447 and serialised at lines 448–449:

```
bundle = dict(clf_fa=clf_fa, clf_wd=clf_wd, le_fa=le_fa,
              le_wd=le_wd, ohe=ohe, tfidf_d=tfidf_d, ...)
with open(MODEL_PATH, "wb") as f:
    pickle.dump(bundle, f)
```

The auto-training mechanism is implemented in `load_models()` (`ml_predictor.py:454-458`):

$$\text{load\_models}() = \begin{cases} \text{pickle.load(MODEL\_PATH)} & \text{if file exists} \\ \text{train\_and\_save}() & \text{otherwise} \end{cases} \tag{128}$$

The model path is defined at line 93: `MODEL_PATH = os.path.join(BASE_DIR, "trace_models.pkl")`. The bundle is loaded lazily via the global `_bundle` variable (line 461), initialised on first `predict()` call (lines 837–839).

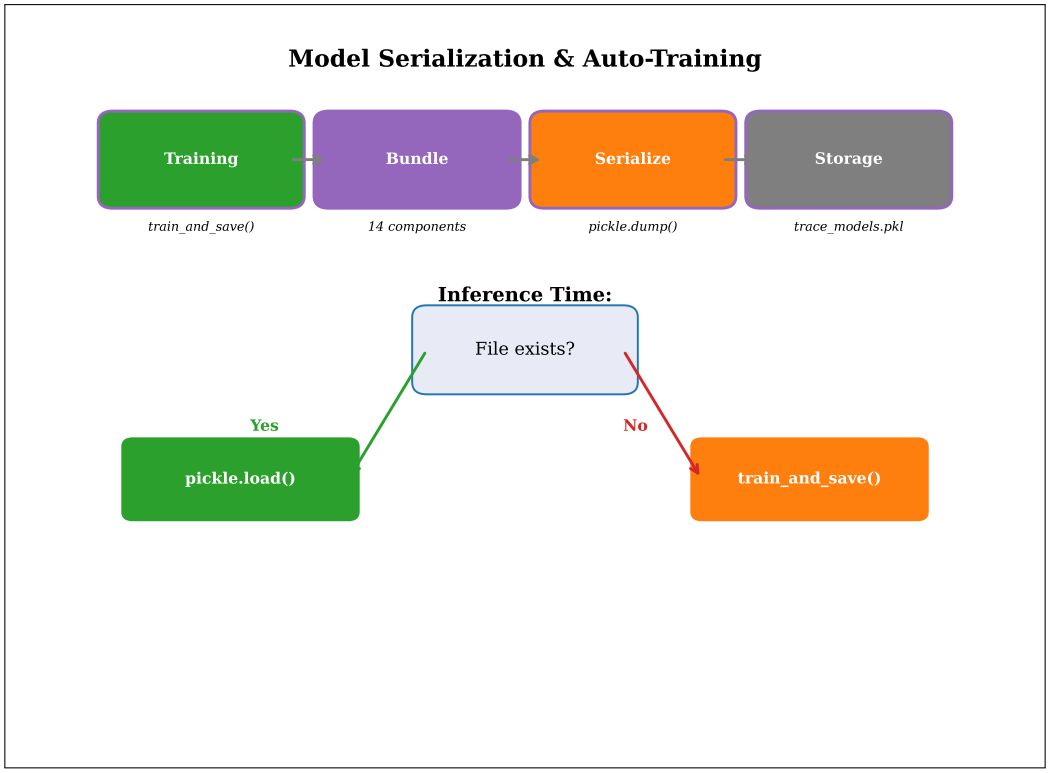

Figure 38: Model serialisation and auto-training showing the training-to-inference lifecycle.

Used in: Section 5.4 (bundle loading for inference), Section 3.4 (bundle creation during training). End of Chapter 4.